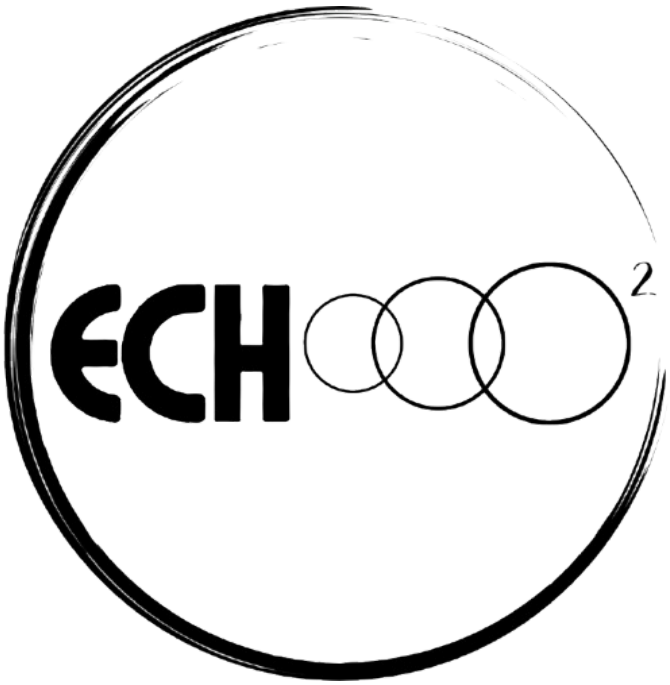


An Interactive Simulation Platform to Explore Information Spread

Team Name: EchoEcho
Proposed Level of Achievement: Apollo 11
Repository Link: https://github.com/valeriecyz/EchoEcho
Deployment Link: https://echo-echo-self.vercel.app/

NUS Orbital 2026

Table of Contents

1. Project Summary **6**
1.1 Project Overview 6
1.2 Problem Motivation 6
1.3 User Stories 6
2. Features and Application Logic **8**
2.1 User Authentication 8
2.1.1 Workflow diagram 9
2.2 Onboarding Tutorial & Guided Walkthrough 9
2.3 Parameterised Scenarios 10
2.4 Interactive Feed 11
2.4.1 Workflow diagram 12
2.5 Spread Visualisation 12
2.6 Critical Thinker Score 13
2.7 Learn More 14
2.8 Replay network generation tick-by-tick 15
2.8.1 Workflow diagram 16
2.9 Leaderboard Ranking 16
2.10 Global Navigation & Session Tools 18
2.11 Saved Simulations – Attempt History 19
3. Design and Architecture **20**
3.1 System Overview 20
3.2 Technology Stack 20
3.3 Architecture Flow 21
3.3.1 High Level Architecture 21
3.3.2 App Folder Structure 22
3.3.3 User Flow 23
3.3.4 Simulation Engine 24
4. Simulation Logic **25**
4.1 Spread Logic 25
4.1.1 Network Architecture and Fundamental Rules 25
4.1.2 Activation Probability 25
4.2 Scoring Formula 27
4.2.1 Evaluation Criteria 27
4.2.2 Mathematical Formula 28
4.2.3 Why 1-B 28
5. Database Design **29**
5.1 Database Technology 29
5.2 Entity Relationship Diagram 29
5.3 Table Schemas 30
5.3.1 Users 30
5.3.2 Posts 30
5.3.3 Attempts 31
5.4 Relationship cardinality 32

5.5 Supporting tables for each feature 33
6. SWE Practices 34
6.1 Version Control 34
6.1.1 Git Workflow 34
6.1.2 Issue Tracking 34
6.1.3 Pull Request Review Process 35
6.2 Code Principles 36
6.2.1 DRY (Don't Repeat Yourself) 36
6.2.2 Separation of Concerns 36
7. Design Pattern 37
7.1 Model-View-Controller (MVC) 37
7.2 Observer Pattern 38
7.3 Command Pattern 38
8. System Architecture 39
8.1 Component Diagram 39
8.2 Use Case Diagram 40
8.3 Class Diagram 41
8.4 Sequence Diagram 42
9. Internal Service / API Reference 43
9.1 Authentication Operation 43
9.2 Login Operation 44
9.3 Posts Import Operation 45
9.4 Fetch Posts Operation 46
9.5 Save Interaction Operation 47
9.6 Save Attempt Operation 48
9.7 Password Reset Operation 49
10. Deployment 50
11. Testing Strategy 51
11.1 Database Validation Testing 52
11.1.1 Posts table schema validation 52
11.1.2 Truth label normalisation validation 53
11.1.3 Virality Range Validation 53
11.1.4 Duplicate detection 54
11.1.5 Publisher validation 55
11.1.6 English content validation 55
11.2 Unit Test 57
11.2.1 Post retrieval service from Supabase 57
11.2.2 Save attempt service 57
11.2.3 Save attempt route action 58
11.2.4 Route protection 59
11.2.5 Filter posts 61
11.2.5.1 Manual testing log 62
11.2.6 MCDA scoring 62
11.2.7 ICM spreading logic 65

11.3 Integration Test 66
11.3.1 User flow and route guards 66
11.3.2 Simulation UI + ICM + MCDA 67
11.3.2.1 Integration Bug and Fixes 68
11.4 User Test 69
11.4.1 Graph visualisation 69
11.4.2 Nodes on graph 69
11.4.3 Share button 69
11.4.4 Explanation 69
11.4.5 True vs false distinction 69
11.4.6 Password reset 69
12. Continuous Integration/ Continuous Deployment (CI/ CD) 70
12.1 CI – GitHub Actions 70
12.2 CD – Vercel 70
13. Security Considerations 72
13.1 Supabase Authentication 72
13.2 Password Hashing 72
13.3 Row Level Security (RLS) 72
13.4 Input Validation 72
13.5 Password Reset Link Security 72
14. Maintenance & Troubleshooting 74
14.1 Known Issues 74
14.2 Maintenance 74
15. Milestone 3 Enhancements 75
15.1 Image Classifier Service Expansion 75
15.2 Forgot Password Flow During User Authentication 75
15.3 Contextual Help Icons On Parameter Configuration 76
15.4 Clearer Onboarding Tutorial 76
15.5 Back Button Improvements 76
15.6 Sign Out Behaviour Improvements 77
15.7 Quit Simulation Without Saving Attempt 77
15.8 Learn More Explainer Cleanup 77
15.9 Cleaner Simulation Feed 78
15.10 Clearer Graph Visualisation 78
15.11 Interface Polish 78
15.12 Post Import Diversity and Feed Variation 79
16. Potential Future Enhancements 80
16.1 AI-Powered Content Curation Pipeline 80
16.2 Interactive “What-If” Counterfactual Learning 80
16.3 Downloadable Summary Report 80
17. Appendix 82
17.1 References 82
17.2 Project Timeline 82
17.3 Contribution Log 83

1. Project Summary

1.1 Project Overview

EchoEcho is a gamified web-based social media simulation designed for users to explore how misinformation spreads in online environments.

It's simple. Navigate a simulated social media feed, like / share / dislike / comment on the post, and visualize your impact on the spread of misinformation.

This project aims to combine behavioural simulation with data visualisation to enable users to better understand the dynamics of online echo chambers and misinformation spread.

At the end of each session, users receive:

- A visualisation of information spread
- A Critical Thinker Score
- Personalised feedback
- Educational explanations for misleading content

1.2 Problem Motivation

In recent years, social media platforms have become major channels for information dissemination. While they allow people to communicate quickly and access information instantly, they also make it easier for misinformation to spread rapidly across online networks.

In a world where everything happens so fast, many users underestimate how everyday social media interactions, such as liking, sharing, disliking, or commenting, can influence what content becomes amplified. These actions may seem small and instant, but each one can affect what viewpoints are repeated, what information reaches others, and how echo chambers are formed.

A key issue is that users are often repeatedly exposed to similar viewpoints, which can reinforce existing beliefs and make misleading or false content seem more familiar or convincing. Over time, this can contribute to the unintentional spread of misinformation.

EchoEcho is motivated by the need to make these hidden effects more visible. By simulating a simplified social media environment using curated real-world posts, EchoEcho allows users to see how their digital choices can affect information spread.

The goal of EchoEcho is to help users build critical thinking skills, recognise echo chambers, and navigate today's information landscape with greater awareness and responsibility. Instead of reacting quickly and impulsively, EchoEcho encourages users to pause, evaluate, and think more thoughtfully before engaging with online content.

1.3 User Stories

1. As a user who wants to understand how information spreads online, I want to interact with a simulated social media feed and observe how my engagement affects content propagation.
2. As a user who completed a simulation session, I want to know how well I did, so I want to see a summary of my performance.

3. As a user who completed a simulation session, I want to review and understand why some posts are considered biased or are misinformation. I want to learn how to identify them.
4. As a user who wants to understand how information spread under different conditions, I want to customise simulation parameters (eg. number of followers and followings I have on my social media account) and replay scenarios so that I can observe step by step how information propagated.
5. As a user new to this web app, I want to have a step-by-step tutorial to guide me through the simulation.

2. Features and Application Logic

Features in this document are organised using the following tags:

- [Proposed]

The intended scope of the feature for the final product.

- [Current Progress]

An account of what has been already built and tested.

2.1 User Authentication

Users register and log in using email and password credentials. Authentication is managed through Supabase Auth, which handles account creation, login validation, session management, and secure access to user-specific data.

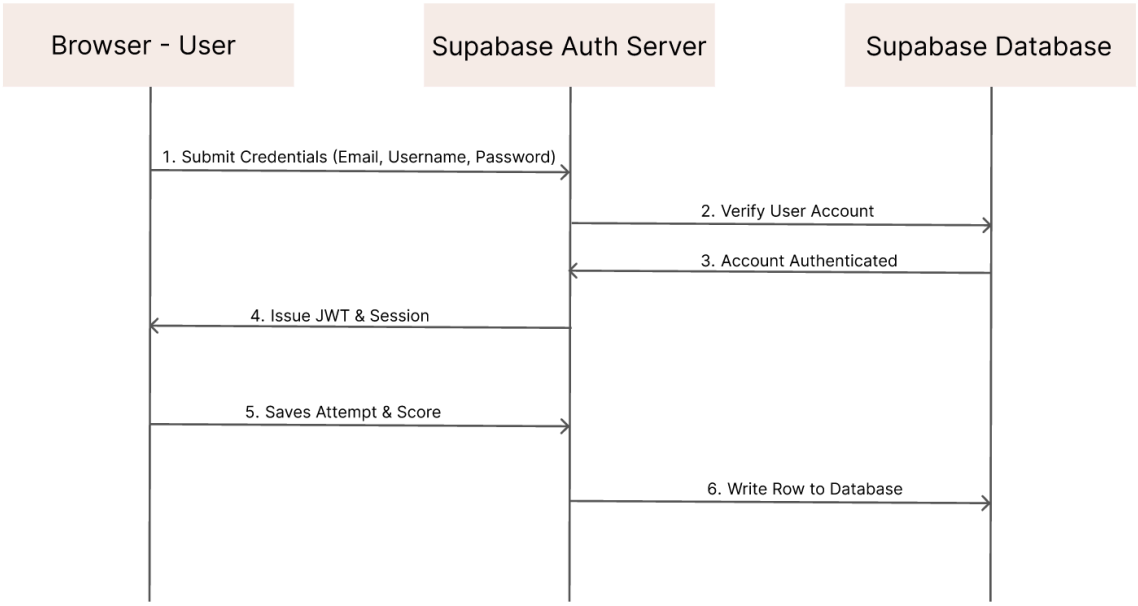
[Proposed]

- Allow users to create an account using email and password
- Maintain authenticated user sessions across visits
- Restrict access to gameplay and score tracking features to authenticated users
- Store user progress, scores, and simulation history under each user account

[Current Progress]

- User registration and login functionality implemented
- User can sign in and access protected pages after authentication
- Supabase backend integration has been completed and tested
- Users can store their scores and attempts under each user account
- Password recovery implemented end-to-end: `/forgot-password` collects user email and calls `supabase.auth.resetPasswordForEmail()`; reset link routes to the `/reset-password/confirm` page, where the user explicitly confirms the reset request via a button click, preventing corporate/ISP email security software (which automatically prefetches links to scan for phishing) from silently invalidating the one-time link before the user clicks it; upon confirmation, `supabase.auth.verifyOtp()` establishes the session and the user is redirected to `/reset-password` to set a new password

2.1.1 Workflow diagram



2.2 Onboarding Tutorial & Guided Walkthrough

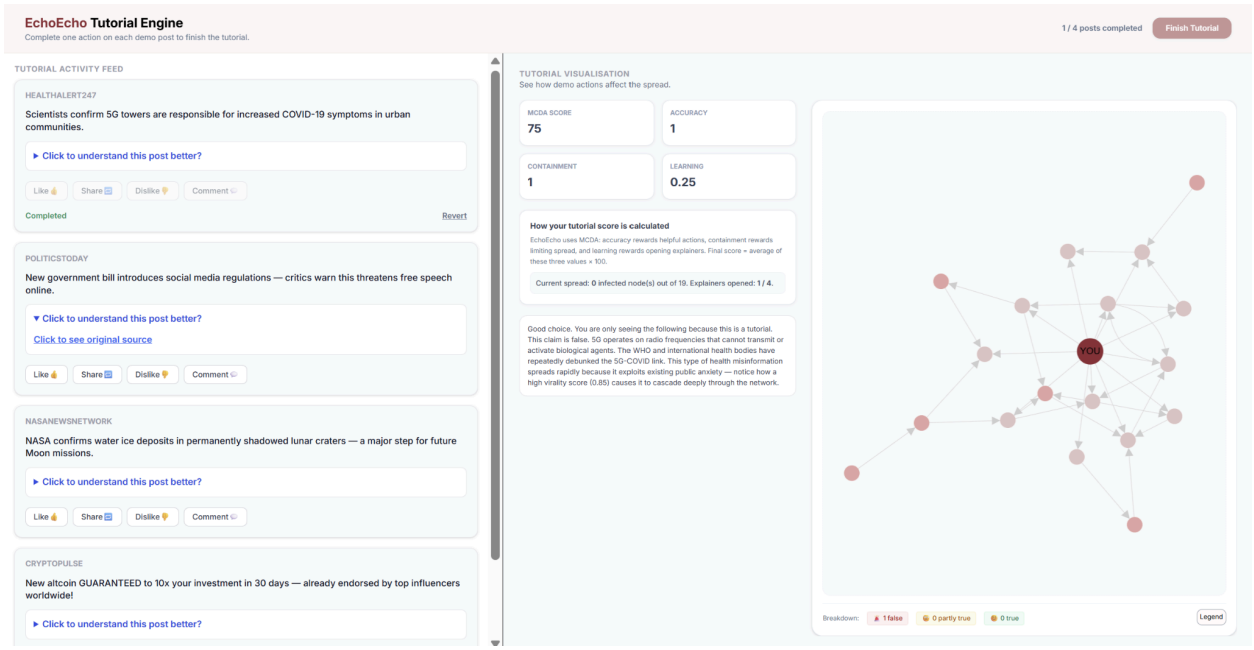
Introduces new users to the objectives of EchoEcho and teaches them how to interact with the simulation environment. This feature has been expanded to also include the contextual walkthrough overlay.

[Proposed]

- Provide first-time users with a guided walkthrough of the platform
- Explain the purpose of the simulation
- Introduce concepts such as information spread, echo chambers, credibility, and critical evaluation

[Current Progress]

- Users are introduced to the simulation concept before entering gameplay
- First time users will go through an interactive onboarding tutorial, guided through a simulation session
- `has_seen_onboarding` column in Supabase tracks first-time-user status
- Guided walkthrough overlay added where each step explains the functionality of a specific UI element
- Walkthrough visibility is tracked per page via session keys combined with a `has_seen_walkthrough` boolean on public.users
- Users can redo the full tutorial in the global navigation menu



2.3 Parameterised Scenarios

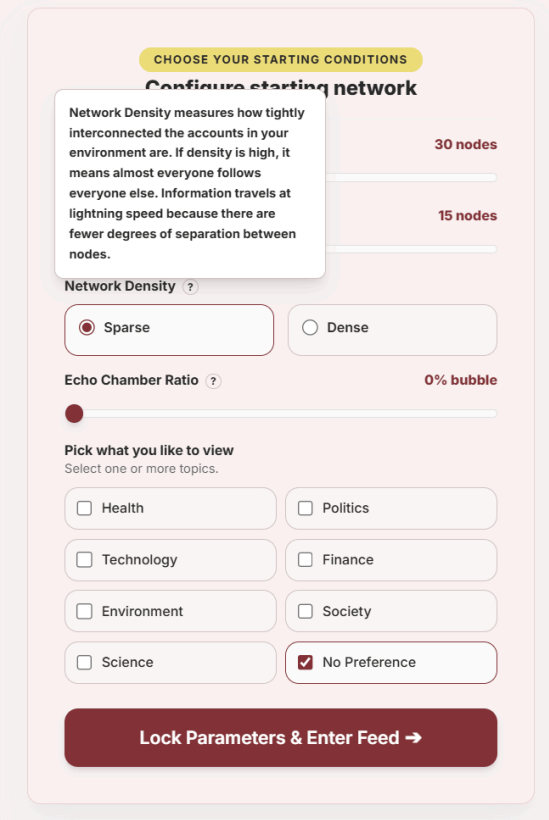
Allow users to customize starting conditions of the simulation.

[Proposed]

- Number of followers
- Number of followings
- Network density

[Current Progress]

- Users configure followers, followings count, network density, Echo Chamber Ratio, and topics before simulation
- Different parameter settings produce different simulated networks and spreading outcomes based on our spread logic (refer to Section 4.1 Spread Logic)
- Verified via testing that chosen parameters are recorded and correctly reflected in the generated network
- Contextual help provided for each parameter



2.4 Interactive Feed

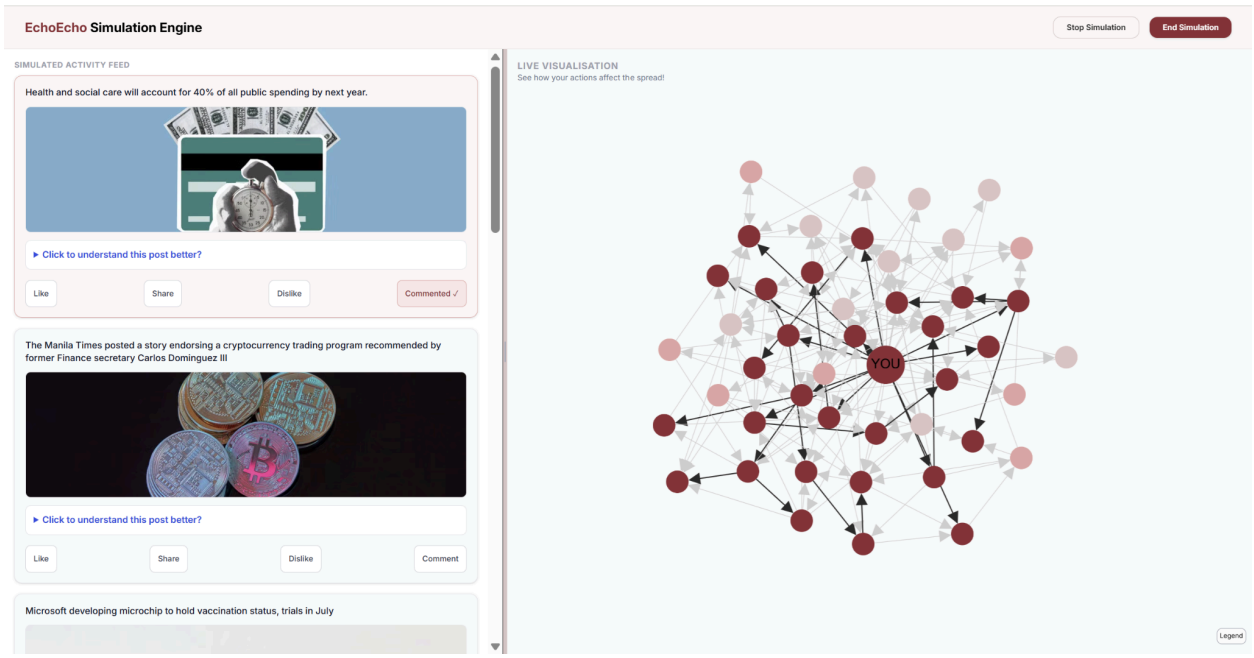
The feed serves as the primary gameplay interface where users encounter posts and decide how to respond.

[Proposed]

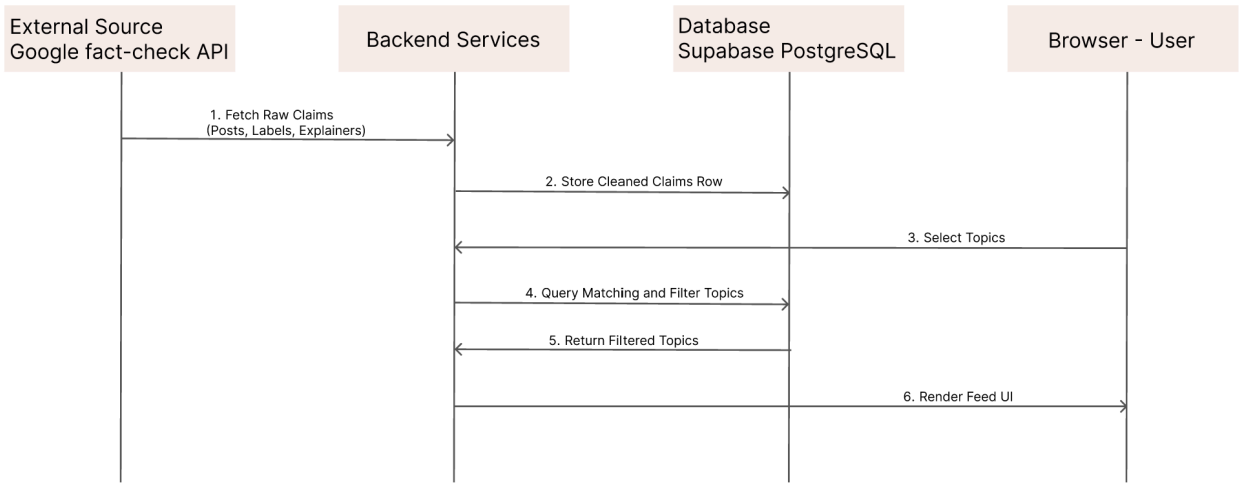
- Users interact with a stream of information posts and choose actions that influence information spread and their score.
- Available actions are Like, Share, Comment, Dislike, Ignore
- Posts vary in topic, difficulty, reliability, potential virality

[Current Progress]

- Feed interface has been implemented: users browse posts and perform Like, Dislike, Share, Comment actions
- Interactions are recorded and network spread updates with each action
- Post content, truth label, explainers and fact-check links are fetched and normalised from Google fact-check Tools API, then stored in public.posts and served based on the user's preferred topics
- Unit testing for fetching and filtering of posts by topics (`db.posts.test.sql`, `fetchPostService.test.ts`, `filter-posts.test.ts`)
- Each action button is individually disabled once used and shows a checked state; Like and Dislike are mutually exclusive, while Share and Comment are each single-use per post
- Post gets a highlighted tint once any action has been taken on it



2.4.1 Workflow diagram



2.5 Spread Visualisation

Visualises how information propagates through a simulated network.

[Proposed]

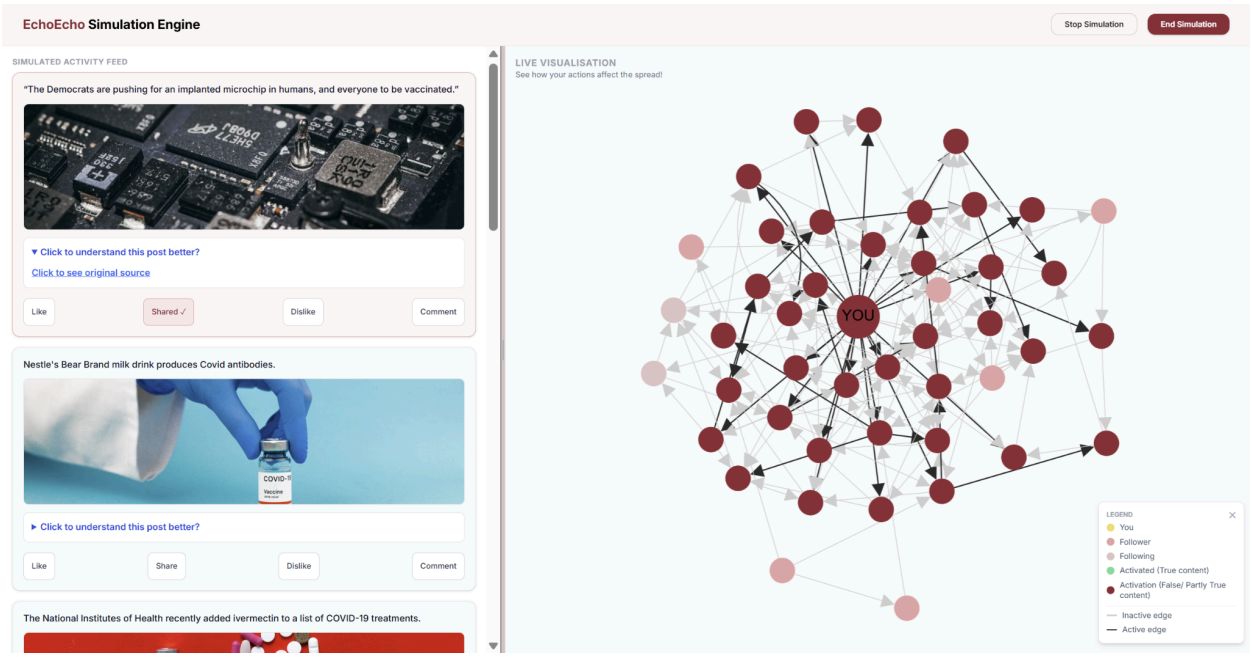
- Display a dynamic network representing users and connections
- Simulate the spread of posts based on user actions
- Colour-code exposure and spread status with animated propagation

[Current Progress]

- Network visualisation implemented
- Simulation network generated from chosen follower/following counts and density
- User interactions trigger simulated propagation: activated nodes turn red
- Core spread logic implemented (Refer to Section 4.1 Spread Logic) and tested.
- Graph legend, distinct follower/following node colours, and reduced node size on dense networks shipped

This simulated network is formed based on:

- Followers Count = 30
- Followings Count = 15
- Network Density = Sparse
- Echo Chamber Ratio = 0



2.6 Critical Thinker Score

Measures a user's ability to engage responsibly with information.

[Proposed]

- Score aims to evaluate user's ability to engage responsibly with information across multiple dimensions including:
    - Accuracy of decisions
    - Appropriate use of Learn More resources
    - Ability to identify misleading content
    - Responsible sharing of content
- Scoring system is designed to reward critical evaluation

[Current Progress]

- Score tracking functionality has been implemented

- User respective interactions with each post contributes to an overall performance score
- Scores are displayed after each gameplay session
- Scoring formula is finalised (Refer to Section 4.2 Score Formula) and tested.
- Implemented score labels categories:

0-29	😈 The Supervillain
30-44	😖 Careless Rookie
45-59	🙈 Neutral NPC
60-74	🔍 Rookie Detective
75-89	😇 Guardian Angel
90-100	🔥 The Super Hero

2.7 Learn More

Provides users with additional context and explanations of each post they interact with.

[Proposed]

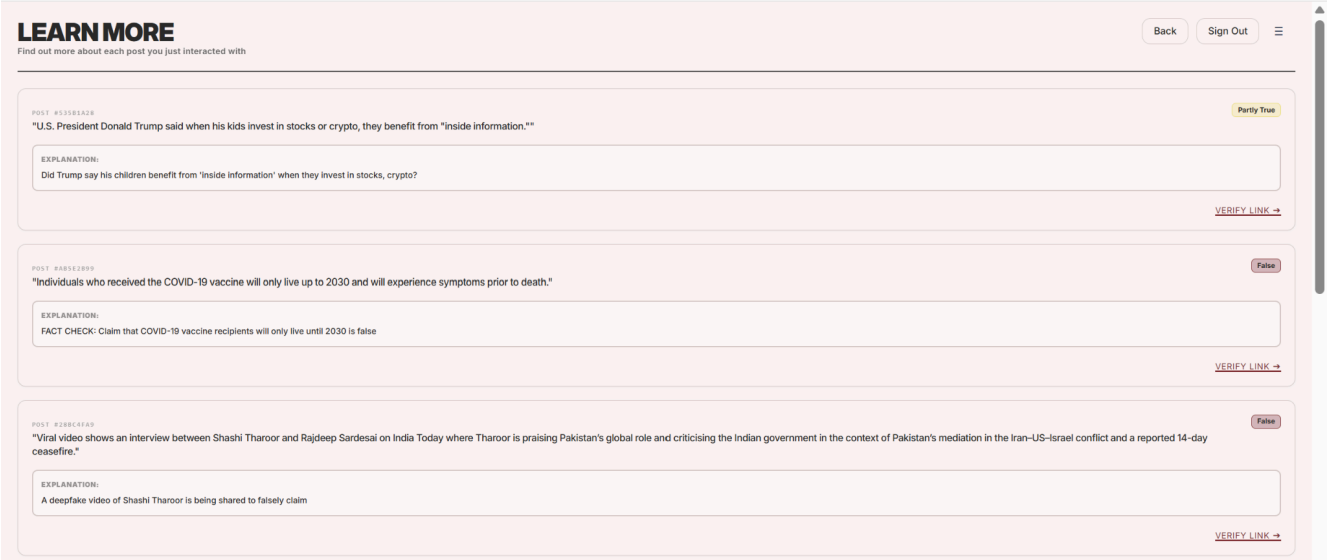
- Allow users to investigate the credibility of a post before making decisions
- Present supporting evidence, fact-checks, and contextual information

[Current Progress]

- "Click to understand this post better?" expands an explainer and links to the original source before the user acts on a post, and opening it feeds Criterion C of the score (refer to Section 4.2 Scoring Formula)
- Screenshot of Learn-more explainer within simulation:



- After the simulation, users can deep dive into each post. Here, users can understand each post's truth label, read the explainers and verify each source via the fact-check link.
- Screenshot of Learn-more after simulation:



2.8 Replay network generation tick-by-tick

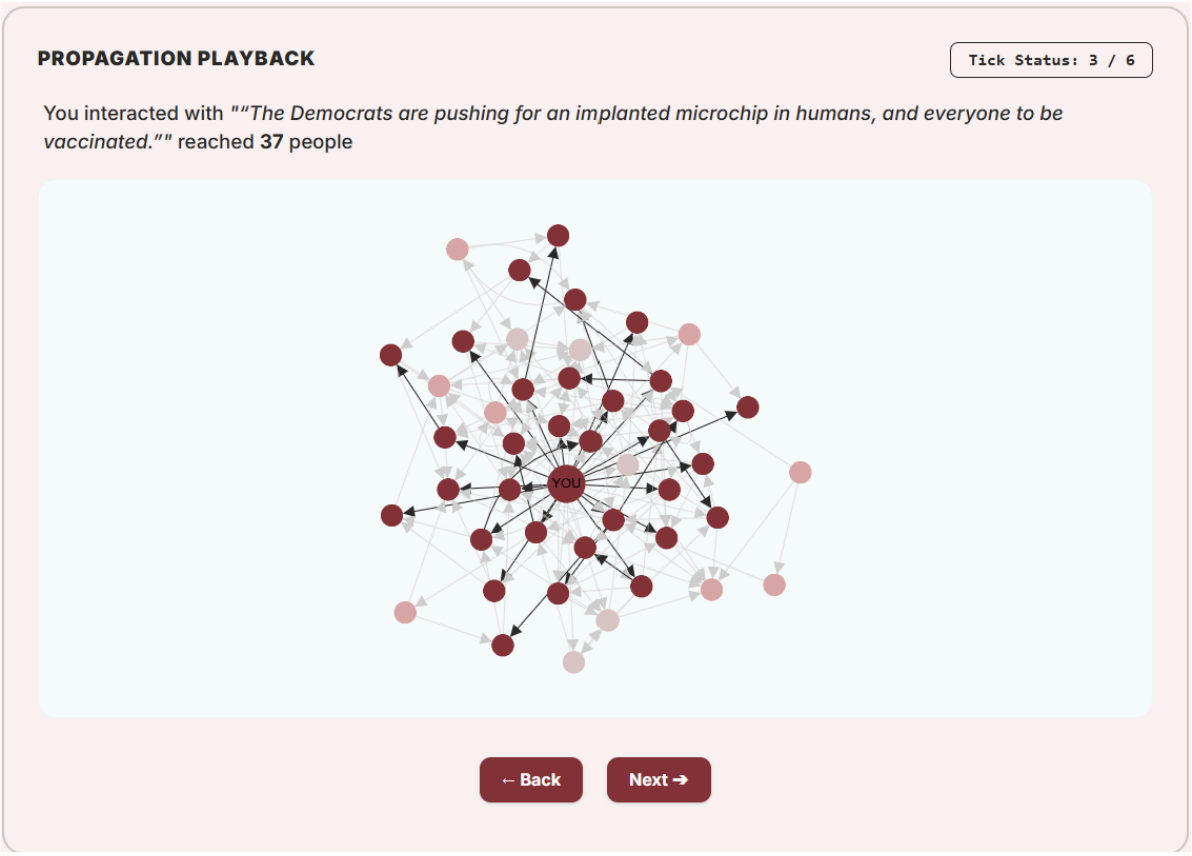
Enable users to view how their network spreads and evolve tick-by-tick.

[Proposed]

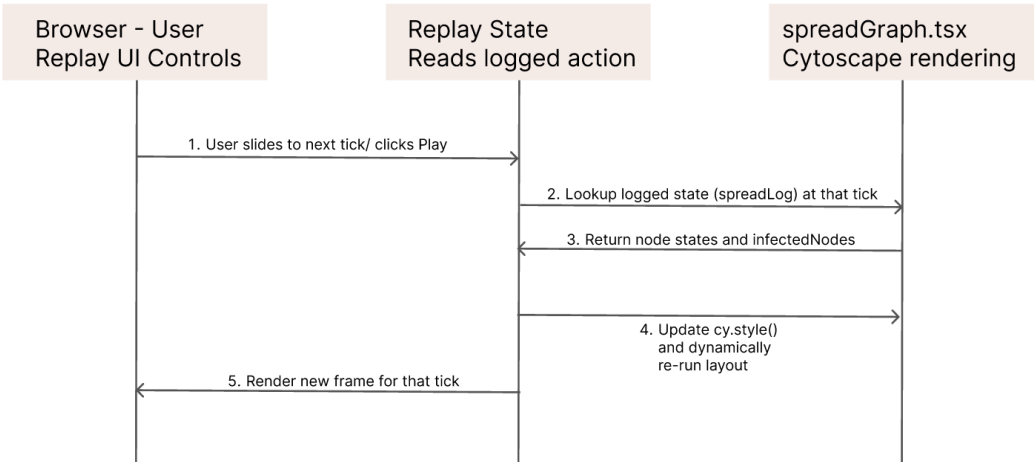
- Step through the simulation timeline incrementally
- Visualise the chronological cascade of information across network nodes

[Current Progress]

- Developed a fully functional timeline replay controller, where users can play, pause, step forward or step backwards to observe the structural changes in their network spread
- Integrated replay state with Cytoscape.js graph rendering
- Mapped the historical logs generated by the spread logic (Refer to Section 4.1 Spread Logic) `spreadLog` to drive the sequential tick-by-tick playback
- Display contextual overlay panel at each tick, stating which user action triggered that specific spread
- Screenshot of current replay function:



2.8.1 Workflow diagram



2.9 Leaderboard Ranking

Provides comparative performance feedback between users.

[Proposed]

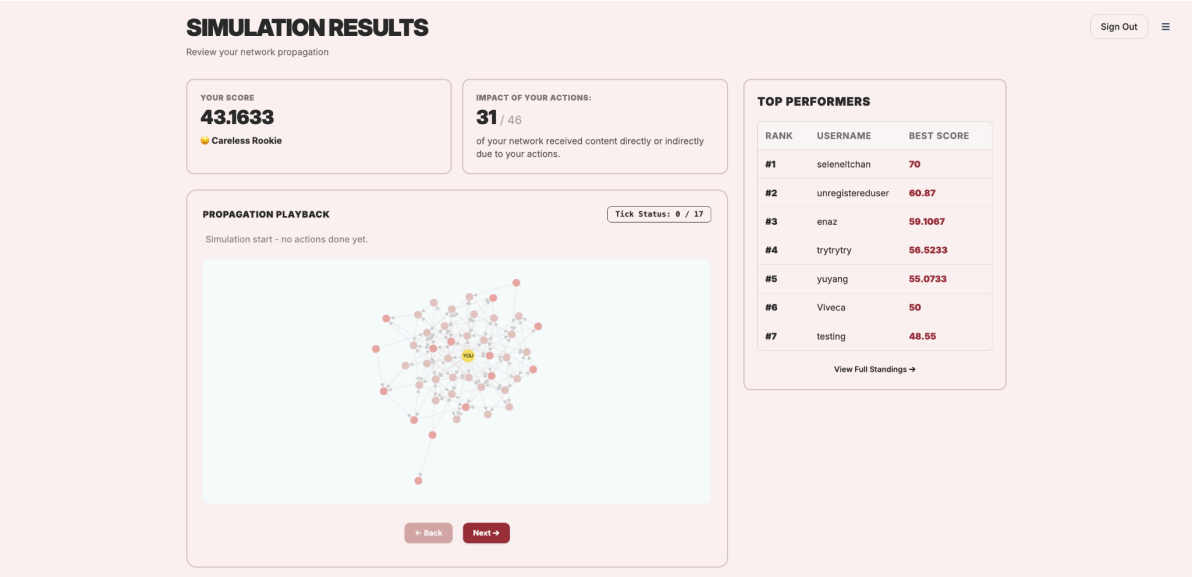
- Rank users based on Critical Thinker Score
- Display top-performing users on a leaderboard

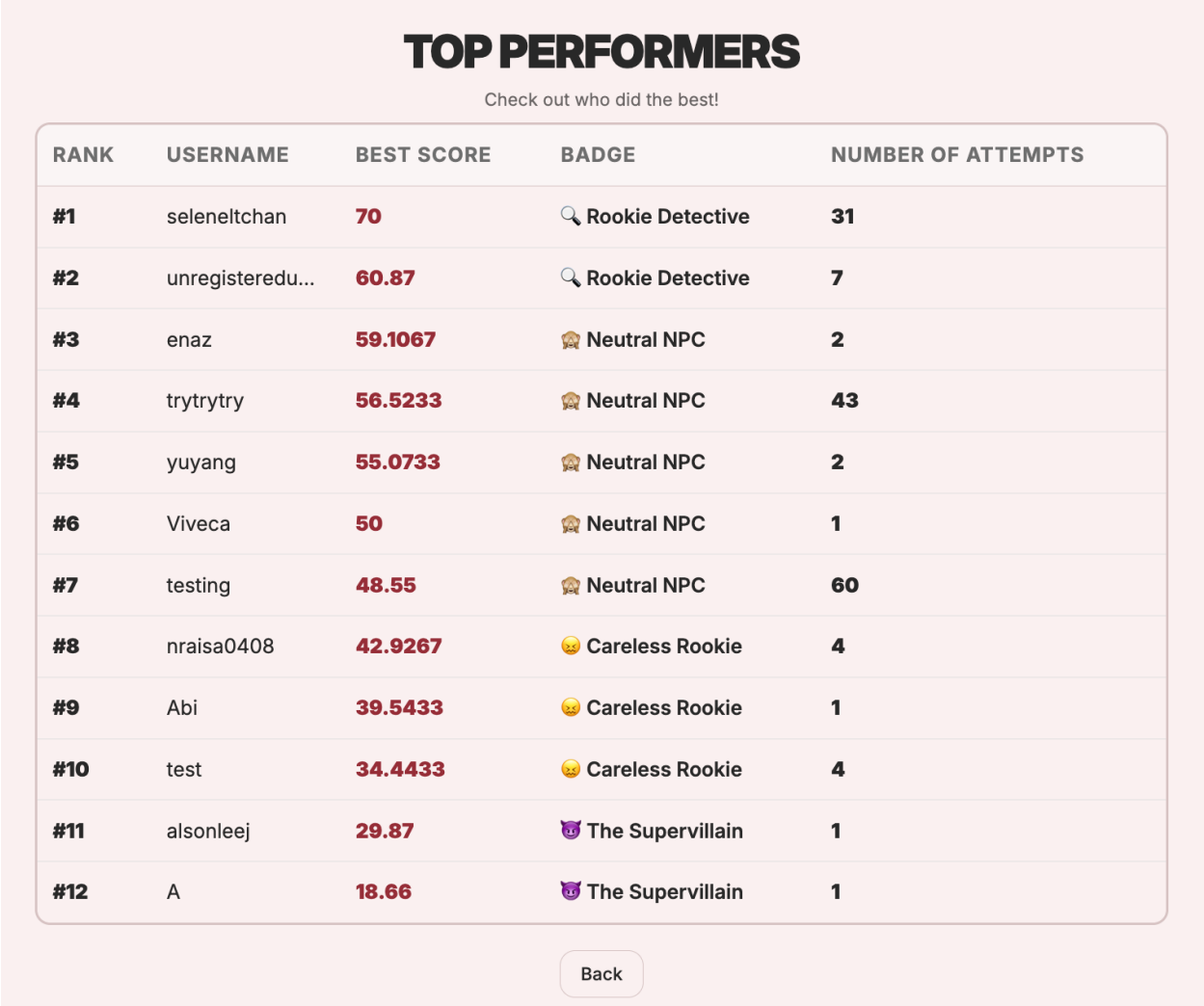
[Current Progress]

- Designed and configured the database schema to handle relational logs linking users to their historical execution attempts and scores
- Built server-side SQL queries that aggregate, sort, and limit data to extract top-performing records dynamically
- A dedicated badge column that shows one of the 6 emoji-tier labels

0-29	😈 The Supervillain
30-44	😖 Careless Rookie
45-59	🙈 Neutral NPC
60-74	🔍 Rookie Detective
75-89	😇 Guardian Angel
90-100	🔥 The Super Hero

- Screenshot of current leaderboard:



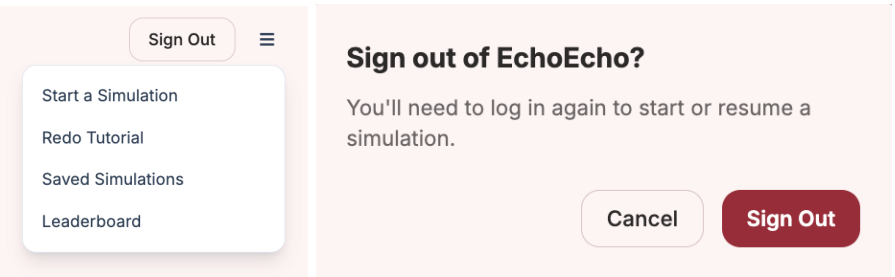


2.10 Global Navigation & Session Tools

Cross-page navigation and session controls used throughout the app.

[Current Progress]

- `NavBar` is a dropdown menu, present on `/home`, `/start`, `/leaderboard`, `/learn-more`, and `/saved-simulation`, linking to a “Start a Simulation”, “Redo Tutorial”, “Saved Simulations”, and “Leaderboard”
- Sign out button opens a confirmation dialog before calling `supabase.auth.signOut()` and navigate to `/`
- Back button calls the browser’s native back navigation when history exists, and falls back to a fixed route when there is no history to go back to

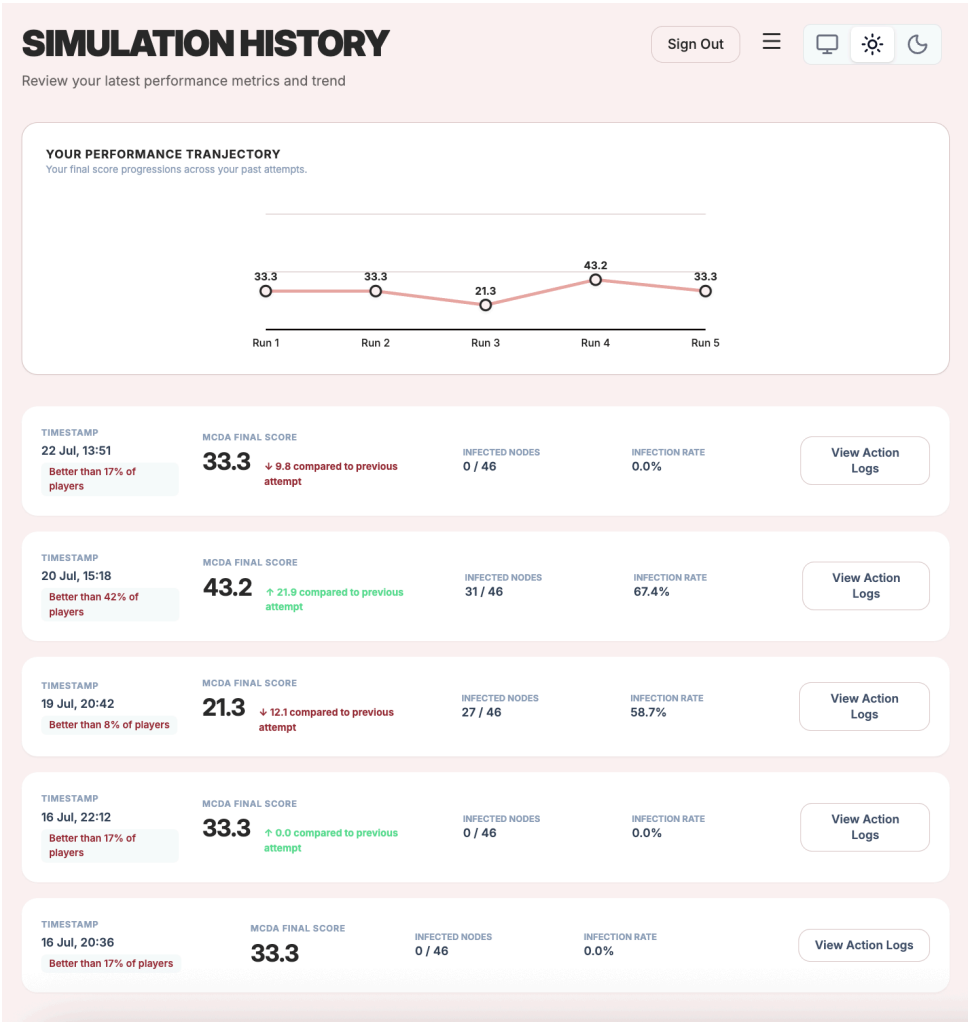


2.11 Saved Simulations – Attempt History

A personal dashboard showing the user's own recent results over time.

[Current Progress]

- `/saved-simulation` route loads the current user's 5 most recent attempts from `public.attempts`
- Each card shows the final score, the change versus the immediately preceding attempt, and the user's percentile among all recorded attempts
- Screenshot of /saved-simulation:



3. Design and Architecture

3.1 System Overview

EchoEcho is a full stack web application designed to simulate how information spreads through social networks and help users develop critical thinking skills when evaluating online content.

The frontend is built using **React** and **Vite**, providing an interactive and responsive user experience. Users can participate in simulation scenarios, interact with posts, view spread outcomes and track their own performance. **Cytoscape.js** is used to create network visualisation, allowing users to observe how their interactions with posts result in propagation of information through different social networks.

Backend is built using **React Router v7**, and **TypeScript service modules**. They handle simulation logic, score computation, post retrieval, and communication with databases. Separating the simulation engine from the frontend allows the spread and scoring logic to be independently maintained, tested, and extended without affecting the user interface.

React Router v7's loaders and actions execute server-side within a **Node.js** runtime, which **Vercel** provisions automatically as serverless functions during deployment.

User authentication and session management are handled through **Supabase**. Application data such as user profiles, simulation results, and leaderboard information are stored in **PostgreSQL** database.

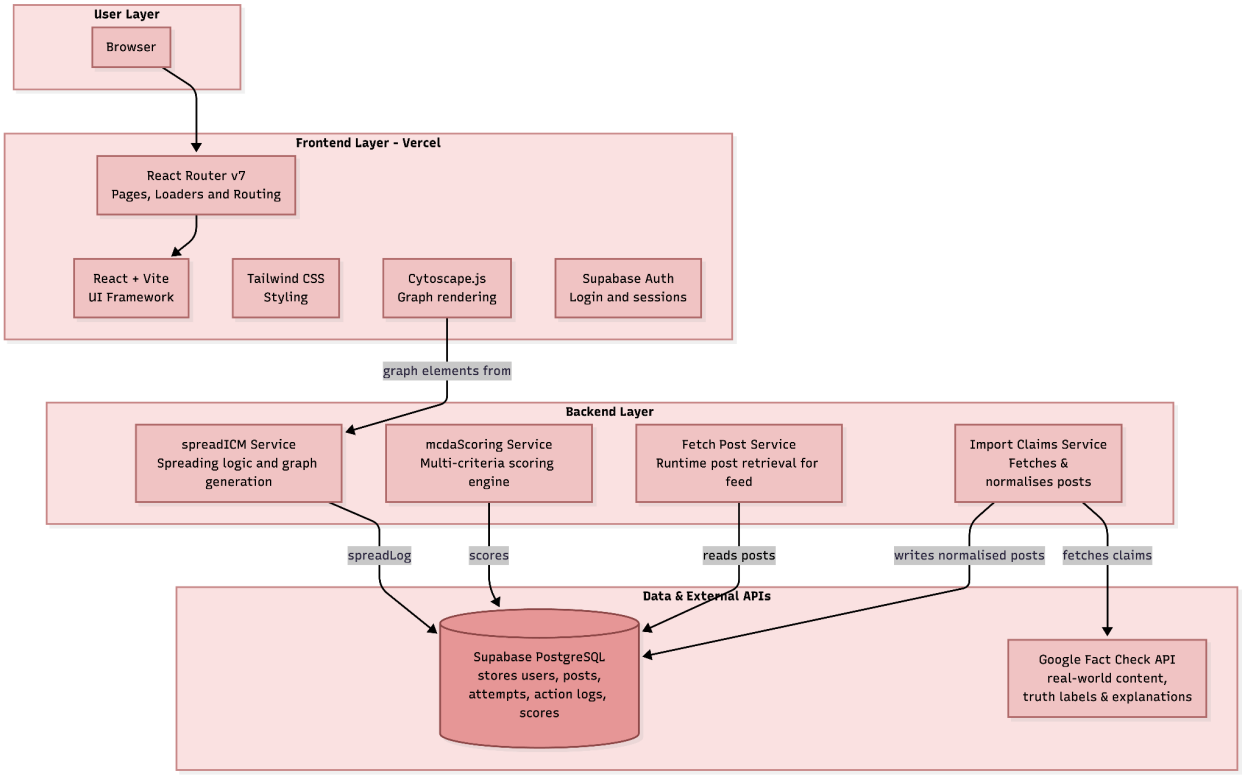
User interface and user flows are designed and prototyped in **Figma**. The application will be deployed and hosted on **Vercel**.

3.2 Technology Stack

Component	**Technology**
Frontend	React + Vite
Graph Visualisation	Cytoscape.js
Styling	Tailwind CSS
Backend	React Router v7 (route loaders & actions) + TypeScript service modules
Runtime	Node.js
User Authentication	Supabase Auth
Database	PostgreSQL (via Supabase)
UI/UX Design	Figma
Version Control	GitHub
Hosting and Deployment	Vercel

3.3 Architecture Flow

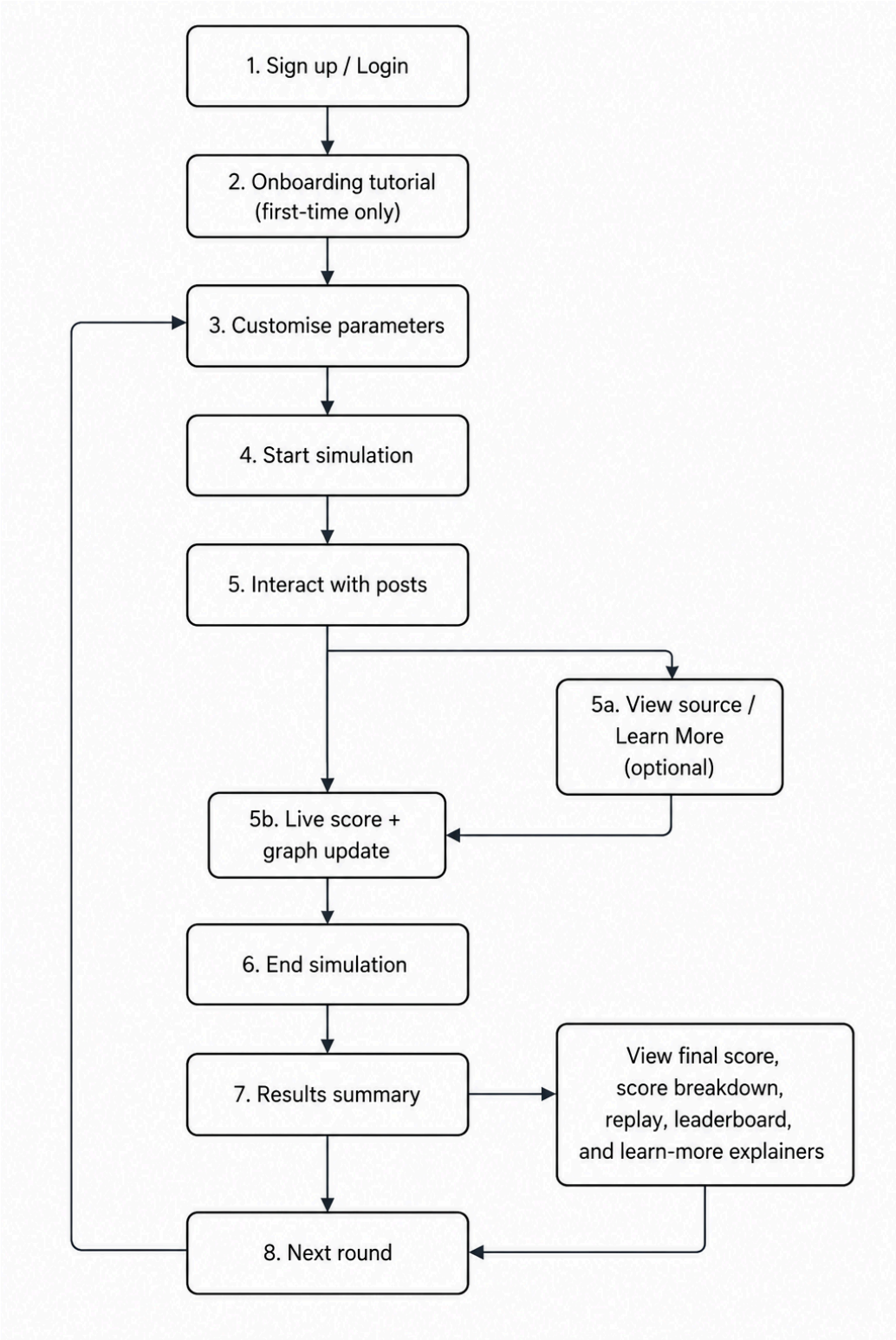
3.3.1 High Level Architecture



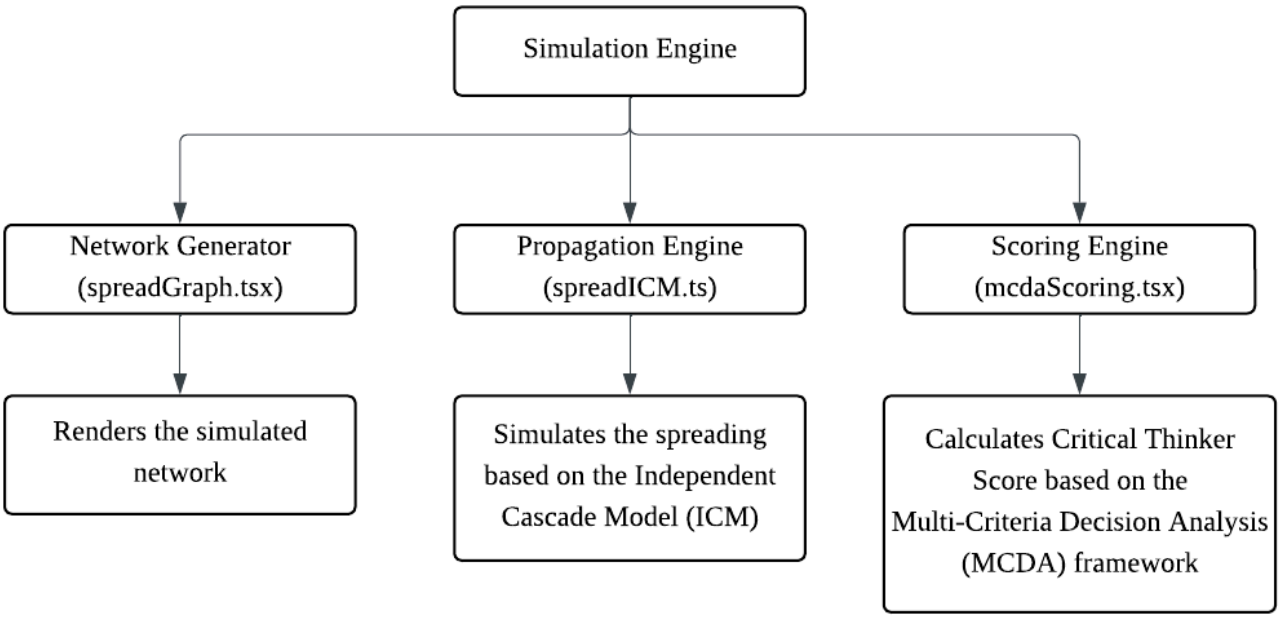
3.3.2 App Folder Structure

```
EchoEcho/
├── app/
│   ├── root.tsx / routes.ts / app.css
│   ├── routes/
│   │   ├── _index.tsx / home.tsx
│   │   ├── login.tsx / signup.tsx / forgot-password.tsx
│   │   ├── confirm-reset-password.tsx / reset-password.tsx
│   │   ├── start.tsx / parameters.tsx
│   │   ├── simulation.tsx / save-attempt.tsx / summary.tsx / saved-simulation.tsx
│   │   ├── leaderboard.tsx / tutorial.tsx / tutorial-replay.tsx / learn-more.tsx
│   ├── components/
│   │   ├── spreadGraph.tsx
│   │   ├── graphLegend.tsx / navBar.tsx / logo.tsx / signout.tsx
│   │   ├── loading.tsx / loading-spinner.tsx / error.tsx
│   │   ├── themeToggle.tsx / walkthrough.tsx / tutorialContent.tsx
│   │   ├── graphElements.ts
│   │   ├── actionLog.ts
│   │   ├── attemptLog.ts / post.ts / tutorial-posts.ts
│   ├── services/
│   │   ├── spreadICM.ts
│   │   ├── mcdaScoring.ts
│   │   ├── postDiversityGate.ts
│   │   ├── fetchPostService.ts / attemptService.ts / leaderboardService.ts
│   │   ├── imageService.ts / classifierService.ts / factCheckService.ts
│   │   ├── topicsHandler.ts
│   └── utils/
│       ├── supabase.ts / supabase.server.ts / supabase.admin.server.ts
│       ├── auth.server.ts
│       ├── themeMode.tsx / styles.ts
│       ├── walkthroughSession.ts
├── scripts/
│   └── importClaims.ts
├── supabase/ (or db/)
│   ├── db_users_schema.sql / db_posts_schema.sql / db_attempts_schema.sql
│   ├── getLeaderboard.sql / has_seen_onboarding.sql
└── tests/
    ├── unit: mcdaScoring_test.ts, spreadICM (icm-model)_test.ts,
    │         postDiversityGate_test.ts, attemptService_test.ts,
    │         fetchPostService_test.ts, imageService_test.ts,
    │         walkthroughSession_test.ts, auth_server_test.ts
    ├── integration: simulation_integration_test.tsx, save-attempt_test.ts,
    │                saved-simulations_test.tsx
    └── e2e: user-flow_e2e_test.ts
```

3.3.3 User Flow



3.3.4 Simulation Engine



4. Simulation Logic

4.1 Spread Logic

The simulation's spread engine is built upon the **Independent Cascade Model (ICM)**, a foundational model used in network theory and social data science to simulate the spread of information, ideas, or behaviors across a network. It treats the propagation of a trend or rumor very similarly to how epidemiologists model the spread of a physical virus.

4.1.1 Network Architecture and Fundamental Rules

The social network is modeled as a directed graph $G = (V, E)$ where Nodes(V) represent individuals (users) and Edges(E) represent lines of directed influence (followers/ following).

The model relies on a few fundamental rules:

1. Binary States: At any discrete tick, a node exists in exactly one of the 2 states: Inactive (unexposed/uninfected) or Active (the individual has interacted with the post and is actively spreading it to their network).
2. Single-Chance Activation: When a node u transitions to the Active state, it gets exactly one single chance to activate its currently Inactive out-neighbors v.
3. Independence: The probability $P(u \rightarrow v)$ that node u successfully activates node v is entirely independent of any prior or simultaneous attempts made by other neighbouring nodes.
4. Discrete Ticks: Time progresses via discrete chronological increments called "ticks". If node u activates node v at Tick t, node v becomes an active spreader starting at Tick $t + 1$. Once a node uses its chance to activate its neighbours, it remains active but can no longer trigger new cascades itself.

4.1.2 Activation Probability

The activation probability $P(u \rightarrow v)$ is determined by 3 interconnected dynamics: the post's virality profile (V), the primary user's interaction with post (A), and the target node's intrinsic susceptibility (S).

$$P(u \rightarrow v) = V \times A \times S, \text{ where } P \in [0, 1]$$

1. Label Normalisation and Post's Virality (V)

To populate the simulation with real content, real-world data is fetched from the Google Fact Check Tools API. The source ratings are parsed and normalised into 3 standardised categories, which directly determine their virality ranges:

Original Ratings	Normalised Truth Labels	Virality Range
False, Misleading, Unsupported, Scam, Conspiracy Theory	False	0.5–1.0 (High baseline spread)
Half True, Mixture, Missing Context	Partly True	0.3–0.7
True, Mostly True, Correct Attribution	True	0.1–0.5

2. User's Action Weight (A)

How the user chooses to interact with each post determines the algorithmic footprint:

User Action	Algorithmic Weight	What it means
Share	0.8	Highest probability. By sharing, the post is forcefully fed into followers' feeds.
Comment	0.5	Moderate probability. By commenting, the user signals high algorithmic engagement.
Like	0.4	Mild probability. By liking, the post's visibility is increased.
Ignore	0.2	Low probability. By ignoring, the user signals neutrality towards that post.
Dislike	0.1	Lowest probability. By disliking, the user signals explicit negative feedback.

3. Target Node's Susceptibility (S)

Target node's susceptibility calculates how vulnerable an individual consumer node is to being activated by an incoming post. It takes into account the node's baseline activity, structural containment within an echo chamber that aligns with the post's topic bias and the incoming post's normalised truth classification.

The calculation of target node's susceptibility is handled by the following function:

```
function computeSusceptibility(node: SimulatedNode, post: SimulatedPost): number {
    // establish baseline based on user activeness
    const base = 0.4 + (node.activeness * 0.3)

    // identify whether in echo chamber
    const postBroad = broadenTopic(post.topic)
    const topicMatch = (
        node.biasTopic !== null &&
        postBroad !== null &&
        node.biasTopic === postBroad
    )

    // apply echo chamber coefficient boost if are in echo chamber
    const echoBoost = (node.inEchoChamber && topicMatch) ? 1.4 : 1.0

    // apply multiplier modifiers based on truth_label of post
    let truthMultiplier = 1.0;
    if (post.truth_label === 'False') {
        truthMultiplier = 1.3;
    } else if (post.truth_label === 'Partly True') {
```

```
        truthMultiplier = 1.1;
    } else {
        truthMultiplier = 0.9;
    }

    // calculate final susceptibility ensuring value is capped within 1.0
    return Math.min(base * echoBoost * truthMultiplier, 1.0)
}
```

4.2 Scoring Formula

To evaluate user performance holistically, we utilize the **Multi-Criteria Decision Analysis (MCDA)** framework. MCDA is a sub-discipline of operations research and decision science designed to evaluate multiple, and often conflicting, criteria to arrive at a structured score or decision.

Rather than simply testing whether a user can guess if a post is real or fake, this scoring engine evaluates the user's ability to evaluate information properly, act responsibly, and demonstrate good digital consumption habits. Hence, the final Critical Thinker Score takes into consideration these factors: Choice Accuracy (A), Network Containment Rate ($1 - B$) and Information Verification Habits (C).

4.2.1 Evaluation Criteria

1. Criterion A: Choice Accuracy

This metric establishes a baseline profile of the user's direct interaction choices. Amplifying truth (for eg. liking or sharing a factual post, ignoring fake news) results in a local value shifting towards 1.0. Doing the opposite drags the local value towards 0.0.

Let P_i be the algorithmic weight of the action taken on post i. Each action is first classified as good, neutral, or bad based on the combination of action type and the post's truth label, then mapped to a quality score $T_i \in \{1.0, 0.5, 0.0\}$. Criterion A is the P_i-weighted average of these quality scores:

$$A = \frac{\sum_{i=1}^{n} (P_i \times T_i)}{\sum_{i=1}^{n} P_i}$$

2. Criterion B: Network Infection Rate

This metric evaluates the real-world cascade consequences of the user's action by looking directly into the ICM simulation spreadLog. It measures the total proportion of the simulated social network that was activated due to the user's specific interaction footprint.

$$B = \frac{number\ of\ nodes\ infected}{total\ number\ of\ nodes\ in\ the\ network}$$

3. Criterion C: Information Verification Habits

This metric reflects the user's digital verification habits. It evaluates whether the user is actively looking into supporting evidence before interacting with the information, instead of reacting based on clickbait hooks.

- During simulation: The “Learn More” explainer panel exposes a verified source_url fetched directly from our normalised database repository.
- Post-simulation review: Each post card exposes the explanation and fact-check link.

$$C = \frac{number\ of\ unique\ explainers\ opened}{total\ number\ of\ available\ explainers}$$

4.2.2 Mathematical Formula

The final score is calculated using the weighted sum model approach, where each criterion holds equal weight, normalised to a maximum value of 100:

$$Critical\ Thinker\ Score = \frac{A + (1-B) + C}{3} \times 100$$

4.2.3 Why $1 - B$

For our scoring system, since a higher final score signifies a good critical thinker, every individual added component must share a positive polarity (In other words, a “better” outcome should yield a larger number).

For criterion B (Network Infection Rate), if a user acts recklessly and spreads misinformation widely, B spikes towards 1.0. Conversely, if a user contains it perfectly, B goes towards 0.0. Therefore, B functions as a minimization metric.

Adding B directly would introduce a logical flaw, where we reward a user with a higher score for destroying the network spread. Hence, we fix this by subtracting B from 1. This inversion shifts the scale properly without breaking the limits:

- Maximum destroy (B = 1.0): If user allows entire network to be infected, the containment component becomes 1 - 1.0 = 0.0 (zero reward)
- Perfect containment (B = 0.0): If user contains misinformation perfectly, the containment component becomes 1 - 0.0 = 1.0 (maximum reward)

Therefore, by doing $1 - B$, the scoring formula ensures that all 3 criteria rewards positive behaviours.

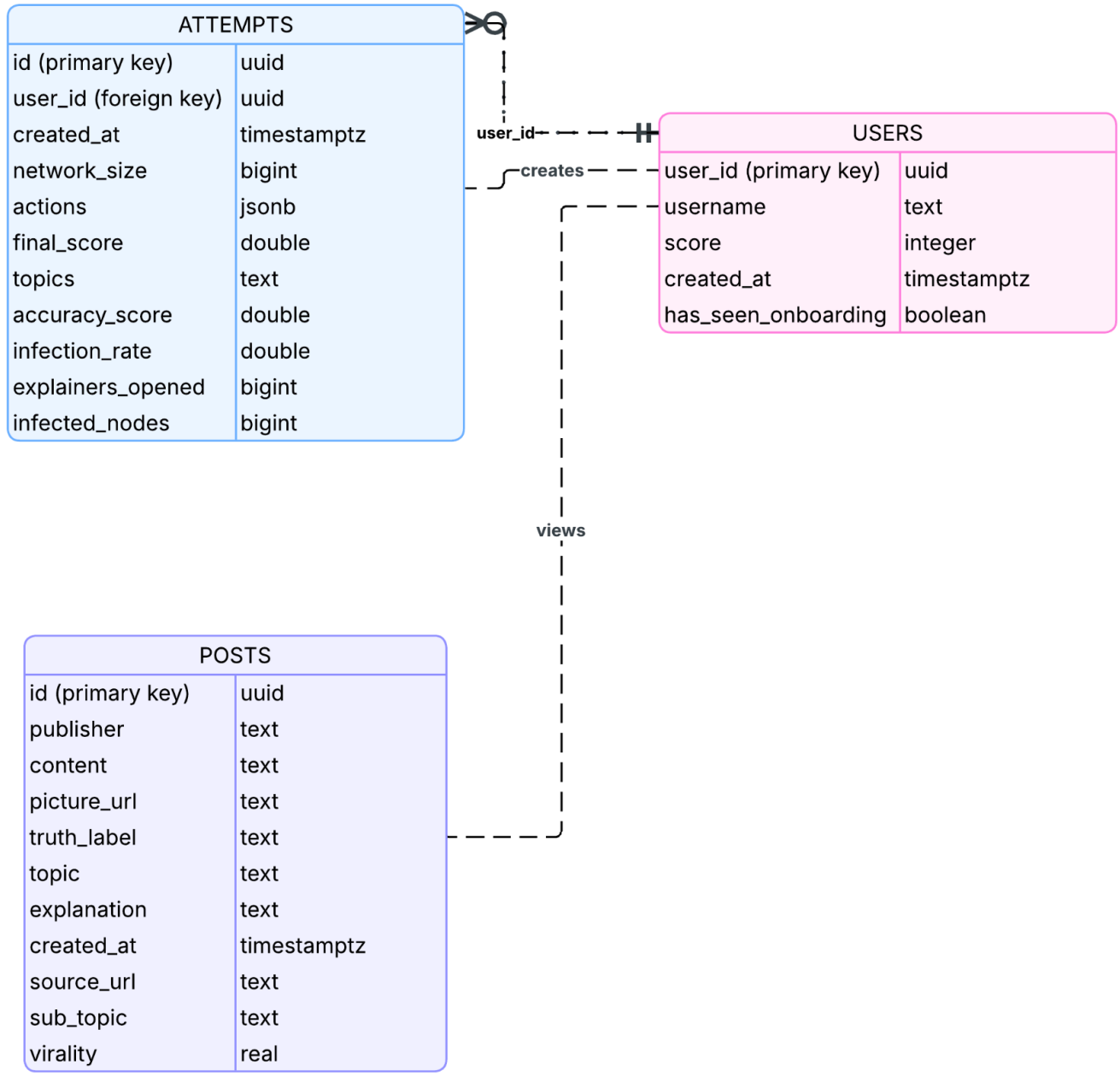
Implementation note: `calculateB()` in `mcdaScoring.ts` returns the containment value directly (equivalent to 1-B above), not the raw infection rate. The raw infection rate stored in `attempts.infection_rate` is reconstructed at the call site as `1 - containmentScore`. Do not confuse the code's internal B variable with this document's B.

5. Database Design

5.1 Database Technology

Supabase PostgreSQL

5.2 Entity Relationship Diagram



5.3 Table Schemas

5.3.1 Users

Table name: public.users

This table is used to:

- Store user information (email, username, completion of the onboarding tutorial, walkthrough status)
- Link to Supabase Authentication through auth.users table

Structure

Column	*Data type*	*Nullable*	*Default*	*Description*
user_id	uuid	No	None	Unique identifier for each user
username	text	Yes	null	Username chosen
created_at	timestamp with time zone	No	current UTC time	Time when user signs up
has_seen_onboarding	boolean	No	false	Whether user has completed the onboarding tutorial
has_seen_walkthrough	boolean	No	false	Whether user has completed the guided walkthrough overlay on the parameters, simulation, summary pages

Constraints

Name	*Type*	*Description*
users_pkey	Primary key	Ensures that each user_id is unique and identifiable
users_username_key	Unique constraint	No two users can have the same username
users_id_fkey	Foreign key	Links user_id to auth.users(id)

5.3.2 Posts

Table name: public.posts

This table is used to:

- Store curated social media-style posts
- Categorise posts by topic and sub-topic
- Provide truth labels for misinformation-related learning
- Store explanations that help users understand why a post may be true/false
- Support scoring through virality value

Structure

Column	*Data type*	*Nullable*	*Default*	*Description*
id	uuid	No	gen_random_uuid()	Unique identifier for each post
publisher	text	No	None	Name of source/publisher
content	text	No	None	Main content
picture_url	text	Yes	null	Image URL assigned to the post
truth_label	text	No	None	Indicates whether the content is true, false, partly true
topic	text	Yes	null	General category
explanation	text	Yes	null	Explanation of truth label to support learning and reflection
created_at	timestamp with time zone	No	current UTC time	Time when the post was added
source_url	text	Yes	null	Optional link to the original source or reference material
sub_topic	text	Yes	null	More specific category
virality	real	Yes	null	How likely the post is to spread in the simulation.

Constraints

Name	*Type*	*Description*
posts_pkey	Primary key	Ensures that each post has a unique id

5.3.3 Attempts

Table name: public.attempts

This table is used to:

- Store completed simulation attempt
- Track user interactions with each post during each simulation
- Record performance metrics, ie. score
- Link every attempt to a specific user record in public.users

Structure

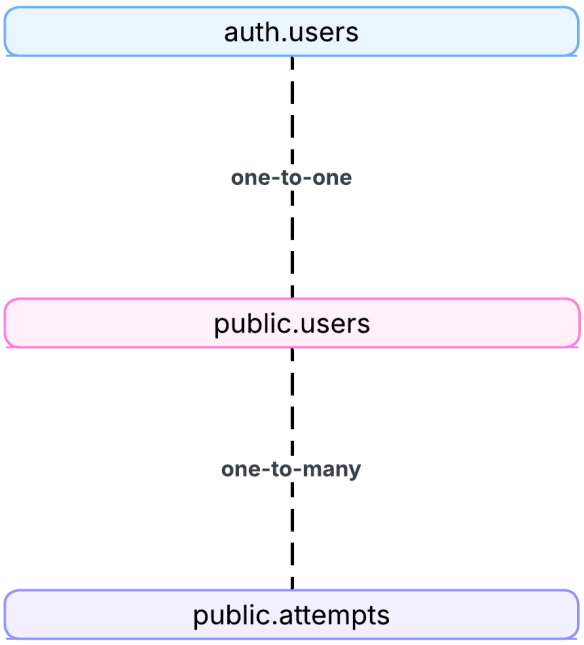
Column	*Data type*	*Nullable*	*Default*	*Description*
user_id	uuid	No	auth.uid()	ID of the user who made this attempt

created_at	timestamp with time zone	No	current UTC time	Time when the attempt was created
network_size	bigint	Yes	null	Size of simulated network
id	uuid	No	gen_random_uuid()	Unique identifier for each attempt
actions	jsonb	Yes	null	User's actions during simulation
final_score	double precision	Yes	null	Final score calculated
topics	text	Yes	null	Topics selected for simulation
accuracy_score	double precision	Yes	null	Accuracy of user's actions
infection_rate	double precision	Yes	null	Ratio of misinformation spread
explainers_opened	bigint	Yes	null	Number of explainers opened
infected_nodes	bigint	Yes	null	Number of infected nodes

Constraints

Name	*Type*	*Description*
attempts_pkey	Primary key	Ensures that each attempt has a unique id
attempts_user_id_fkey	Foreign key	Links each attempt to a user in Users table

5.4 Relationship cardinality



Each authenticated user has one matching user record

One user can have zero or more simulation attempts

5.5 Supporting tables for each feature

Feature	*Supporting table(s)*	*Explanation*
User authentication	auth.users + public.users	Supabase Auth stores sign up and log in records in auth.users while public.users stores app user data
Onboarding tutorial	public.users	has_seen_onboarding field indicates whether a user has already completed the tutorial
Parameter configuration	public.posts + public.attempts	Provides topics and sub-topics for simulation and stores user's selected topic + network size for each attempt
Simulation feed	public.posts	Stores curated posts shown in the feed
Graph visualisation	public.attempts	Attempts data is used to generate the graph
Learn more explainer	public.posts	explanation and source_url provide explainers for each post
MCDA scoring + leaderboard	public.attempts + public.users	Stores result metrics and links to each authenticated user
Replay tick-by-tick	public.attempts	Stores user's interactions during simulation

6. SWE Practices

6.1 Version Control

6.1.1 Git Workflow

Feature Branch Strategy

Instead of committing directly to main, each major feature and test was developed in its own separate branch. This helps us maintain good version control in the actual app.

Workflow followed:

1. Create GitHub issue
2. Create a feature branch
3. Implement the feature
4. Run tests
5. Open a Pull Request, link Pull Request to relevant issues
6. Peer review and status check
7. Merge into main
8. No direct commits to main

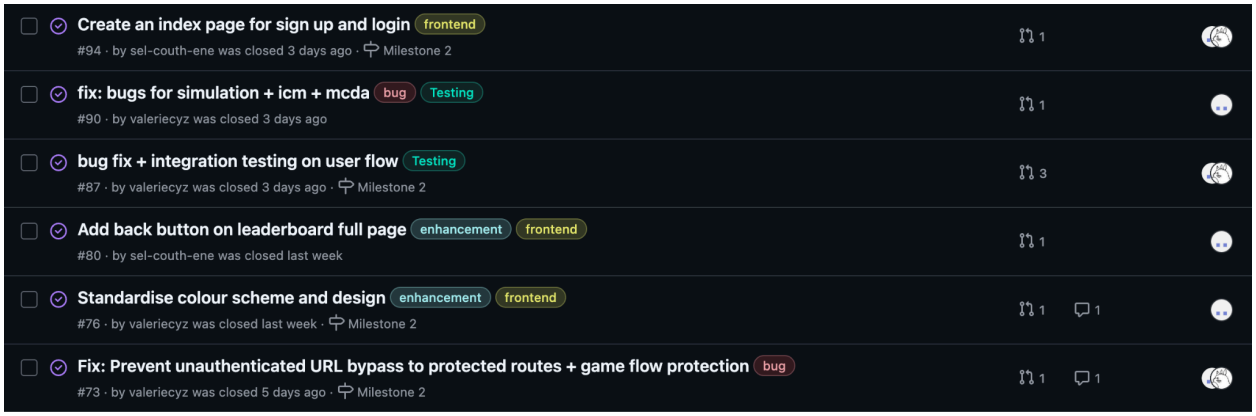
Branch type	*Example branches opened*
Feature	feature/tutorial, feature/replay, feature/leaderboard, feature/feed, feature/intro-page etc.
Testing	test/integration-simulation, test/auth-server, test/icm-model, test/flow-protection, test/scoring etc.
User flow	user-flow, e2e/user-flow etc.
UI	ui/style-cleanup etc.
Bug-fix	bug/73 etc.
Deployment	deployVercel
Revert	revert-66-feature/leaderboard

6.1.2 Issue Tracking

GitHub Issues used for:

- Feature requests
- Bug tracking
- Documentation tasks

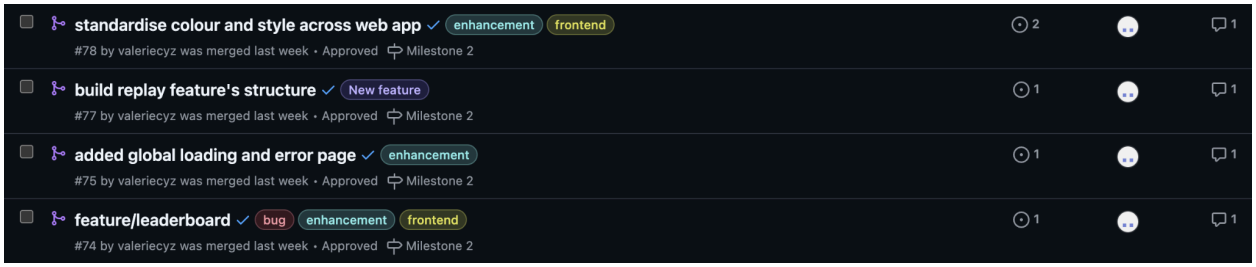
Evidence of issue tracking:



6.1.3 Pull Request Review Process

All code changes require:

- Pull Request
- Approval from teammate



6.2 Code Principles

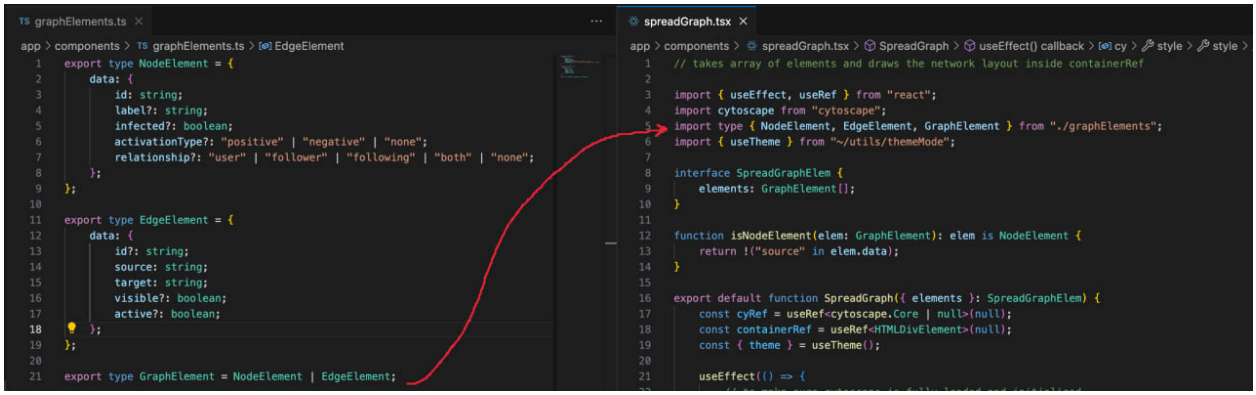
Adhering to the following principles keeps our application easy to maintain and simple to test. The 2 primary principles used were DRY and Separation of Concerns.

6.2.1 DRY (Don't Repeat Yourself)

DRY focuses on reducing repetition by abstracting shared logic into reusable blocks.

How it was implemented:

- The mathematical logic for ICM and the MCDA Scoring calculations are written as centralized and reusable service files (`app/services/spreadICM.ts` and `app/services/mcdaScoring.ts` respectively). The main simulation uses them to run the live simulation and the test suites import these service files.
- Core data contracts (such as `NodeElement, EdgeElement, GraphElement`) are defined exactly once in shared TypeScript definitions files. This ensures that if a data property changes, it instantly and safely updates across the frontend components, backend services and the testing files.



6.2.2 Separation of Concerns

Each section addresses a different problem. A section should only care about its own responsibilities, and then interact with other layers.

How it was implemented: refer to Section 3.3.2 App Folder Structure

- Our application isolates the visual presentation (frontend UI), mathematical scoring and spreading logic.
- The visual interface (stored in `spreadGraph.tsx`) uses Cytoscape.js for network graph visualisation and Tailwind CSS for layout and styling. This layer is responsible for UI elements and responsive design. It does not know how the scoring and spreading works or whether an action is logged.
- The ICM propagation engine (stored in `spreadICM.ts`) handles the graph spreading logic, while the MCDA scoring engine (stored in `mcdaScoring.ts`) handles only the calculation of scores.
- A combination of React states (within `simulation.tsx`) for UI renders, alongside useRef and useEffect tracking hooks used to maintain immediate and synchronous data pipeline. This is isolated from the mathematical logic and calculations.

7. Design Pattern

Design patterns are reusable solutions to a commonly recurring problem within a given context in software design, in particular, how individual pieces of code are structured and interact at a higher level.

Each of the design patterns below emerged naturally from a real problem.

7.1 Model-View-Controller (MVC)

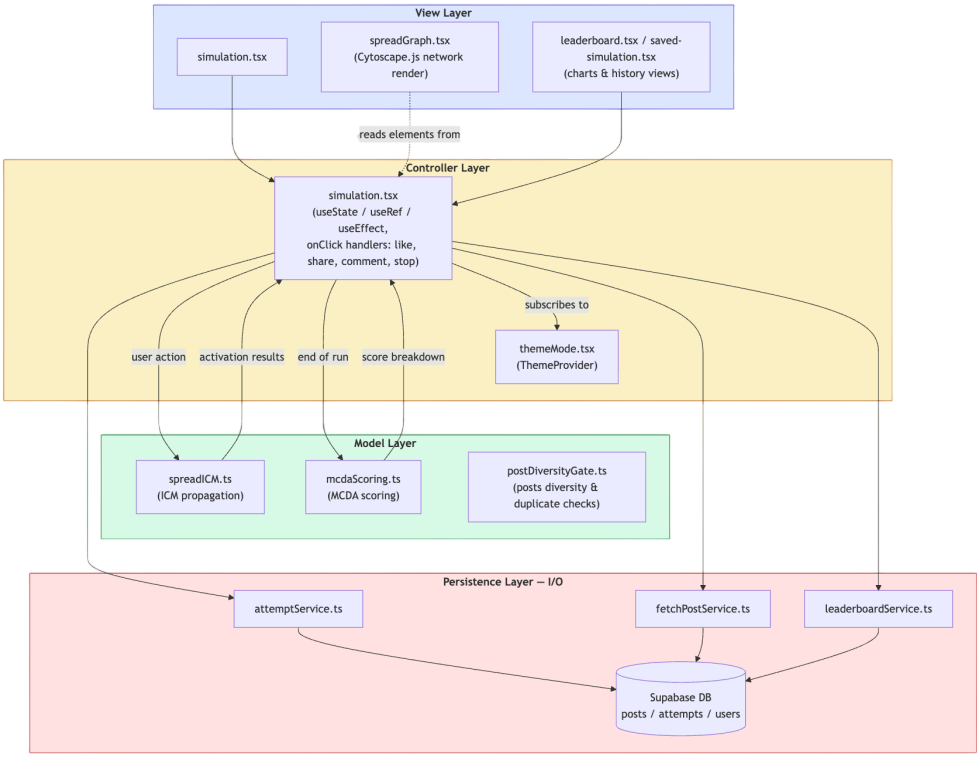
Model: Stores and maintains data & performs computation, updates the view if necessary

View: Displays data, interacts with the user, and pulls from the model if necessary

Controller: Detects UI events (mouse clicks, button pushes) and takes follow up action, including calling the model, then updating the state that flows back into the view

How it was implemented:

- **Model**: `spreadICM.ts` and `mcdaScoring.ts` → ICM propagation math and MCDA score formulas, and return plain data with no rendering logic
- **View**: `spreadGraph.tsx` → takes a plain `GraphElement[]` array and renders it via Cytoscape.js
- **Controller:** `simulation.tsx` → `onClick` handlers detect user events, call into the Model, and push results back into React state, where the View then re-renders from
- Implementation note: the Controller and View are physically the same file (`simulation.tsx`). This is an adapted MVC rather than a classic three-file MVC



7.2 Observer Pattern

Observers subscribe to a subject and get notified automatically when its state changes, without polling.

How it was implemented:

- **Observer:** `ThemeProvider in themeMode.tsx` → call `useTheme()` → subscribing to the theme context and automatically re-render when `mode`/`resolved` changes
- In `themeMode.tsx`, an **OS-level observer** is registered: `window.matchMedia("(prefers-color-scheme: dark)").addEventListener("change", handleChange)` → subscribing to the operating system's theme changes and reacting only when notified
- In `spreadGraph.tsx`, `useEffect(() => {...}, [elements])` re-syncs the Cytoscape graph whenever the `elements` prop changes

7.3 Command Pattern

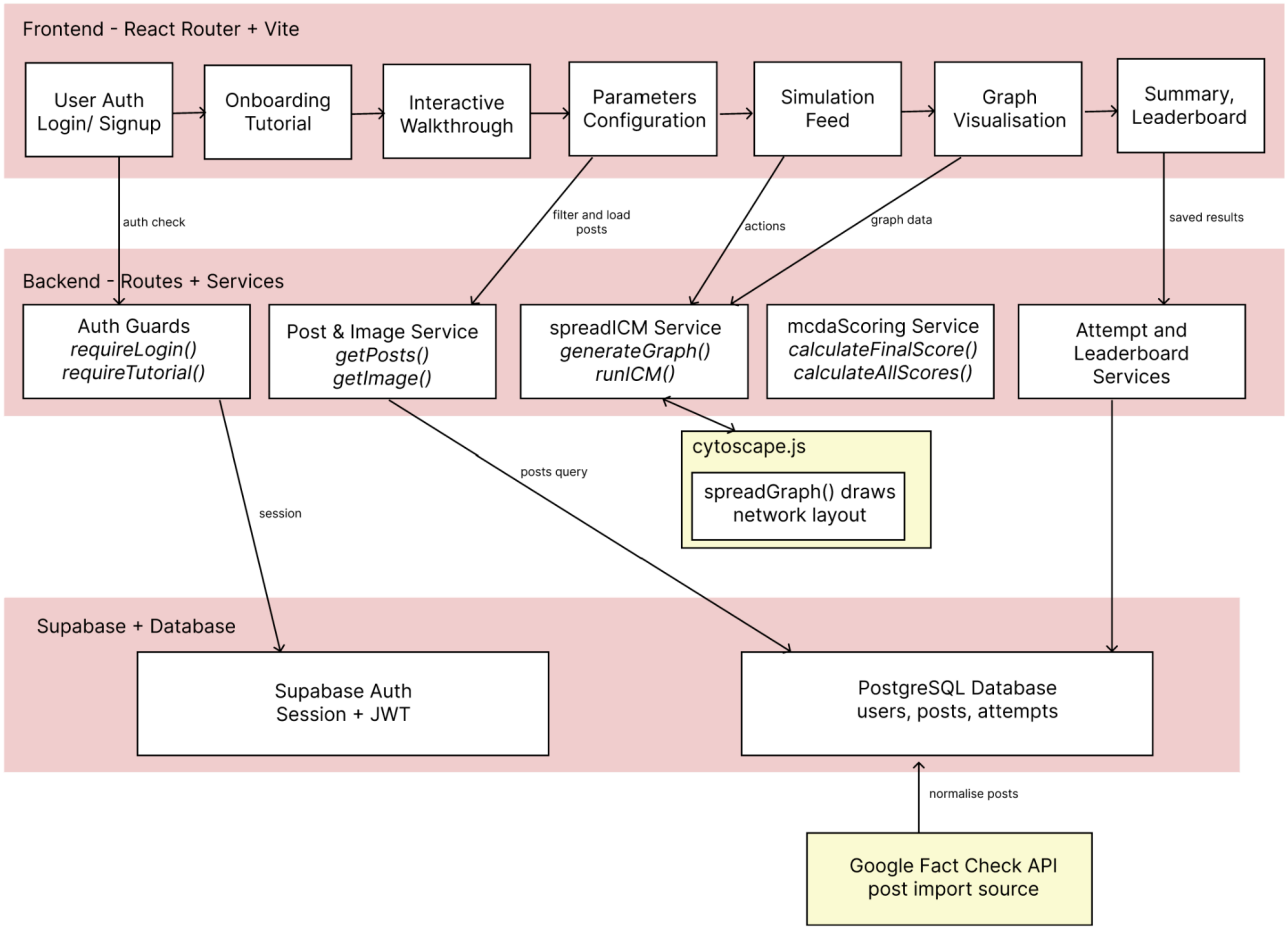
The Command pattern encapsulates a request as a standalone object, so it can be stored, passed around, or replayed independently of the code that triggered it.

How it was implemented:

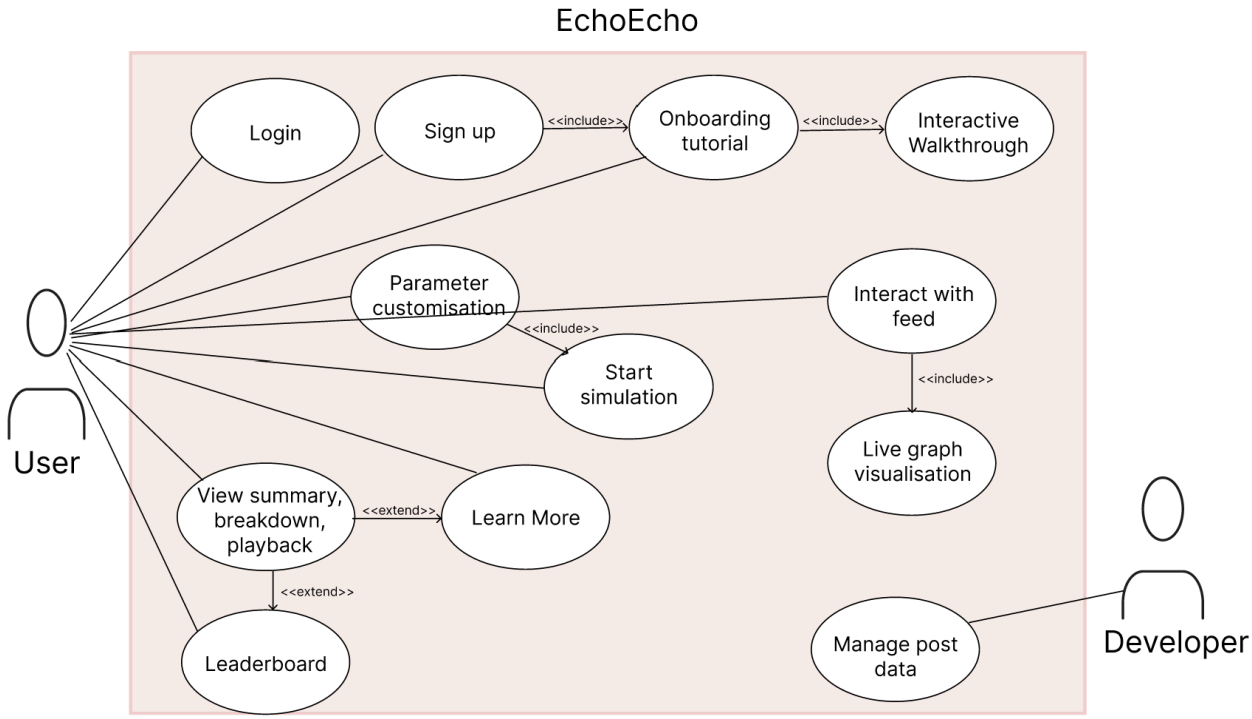
- Every user interaction with a post during a simulation (like, dislike, share, comment) is captured as an `actionLog` object in `app/components/actionLog.ts` in the form of `{ postId, postContent, actionType, impact, commentText, timestamp, ... }`. This bundles the action and its context into one self-contained record
- These objects are stored in an array as an `actionLogs` state in `simulation.tsx`, persisted via `attemptService.ts` into the `attempts.actions` JSONB column, later replayed in `saved-simulation.tsx` action-log viewer to reconstruct exactly what happened
- Implementation note: this is not a classical Command object, it is a data-only command variant

8. System Architecture

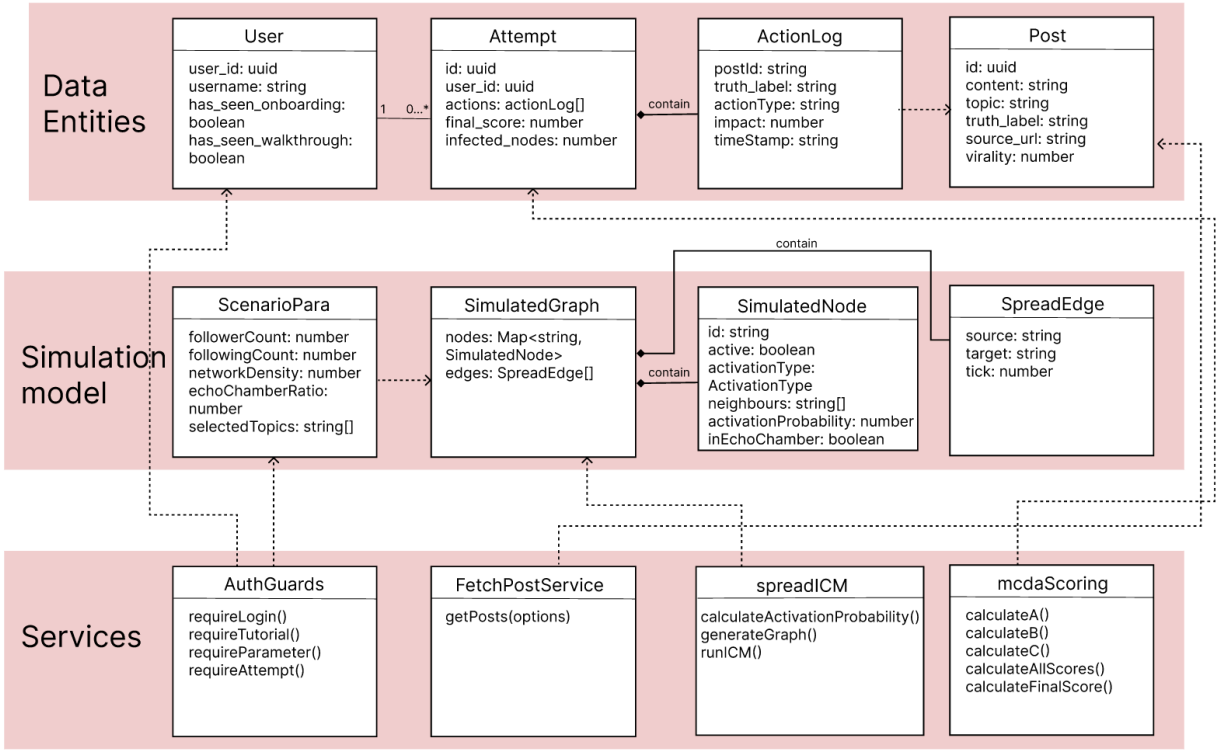
8.1 Component Diagram



8.2 Use Case Diagram

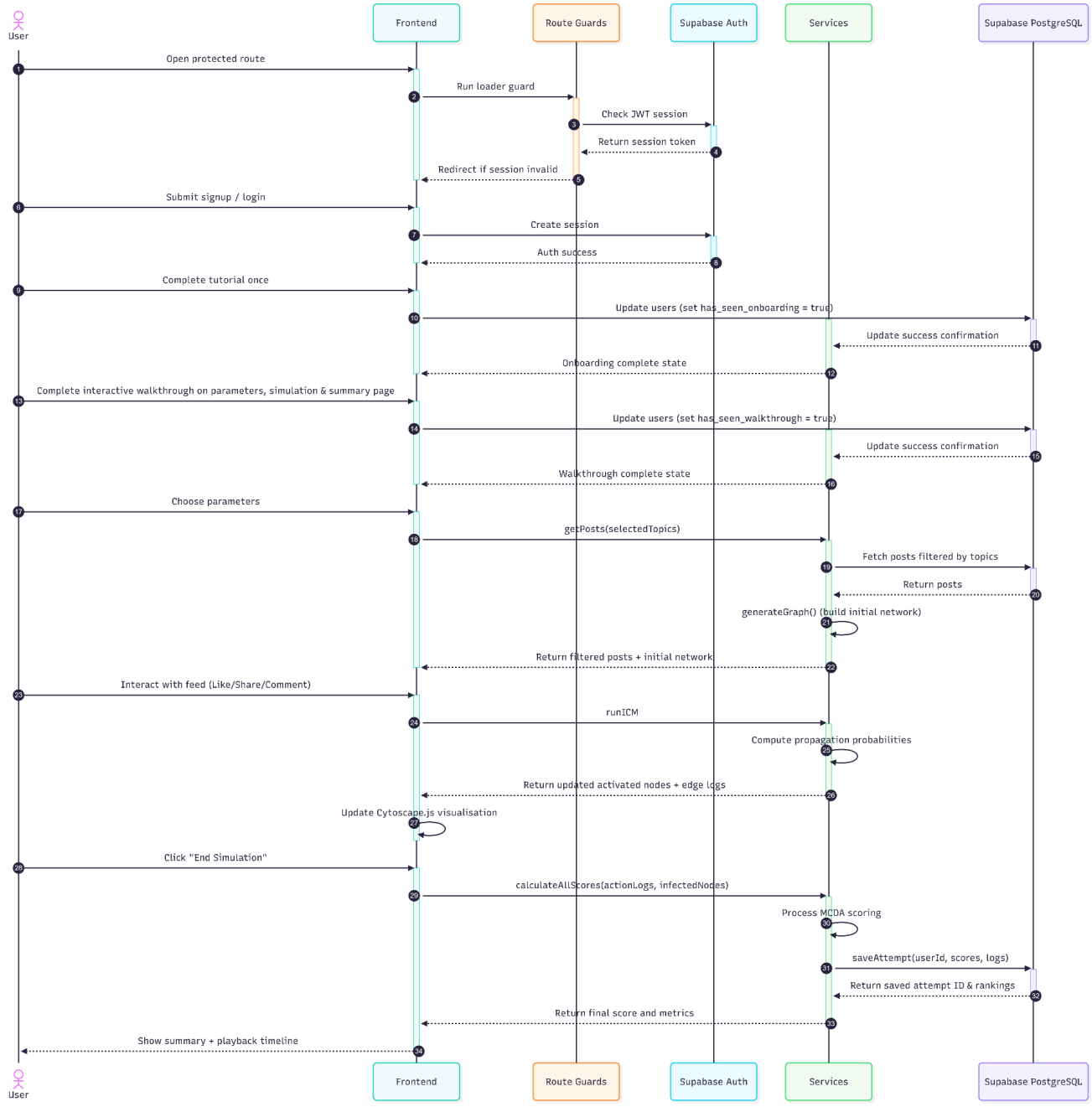


8.3 Class Diagram



Symbol in class diagram	**Meaning**
Solid line with filled diamond attached to parent	Composition (eg. Attempt has an `actionLog`, `SimulatedGraph` has a `SimulatedNode`)
Dashed arrow pointing to data being used	Dependency (eg. `spreadICM` uses `SimulatedGraph` data)
Line with 1 and 0…* written at ends of line	Shows quantities (eg. 1 user can have 0 to many attempts)

8.4 Sequence Diagram



9. Internal Service / API Reference

This app does not rely on traditional standalone REST API servers for every feature. Instead, it mainly uses React Route loaders/actions, service files, and Supabase database calls to handle data fetching, authentication, simulation actions, and result saving.

Therefore, the GET and POST operations in the following section describe the logical API behaviour of the system. In the actual codebase, these behaviours are reflected through route loaders, actions, and service functions.

9.1 Authentication Operation

We use Supabase Auth to handle user sign up, log in, sessions, and protected routes.

This is reflected in:

auth.server
requireLogin()
requireTutorial()
requireParameter()
requireAttempt()
requireOnboarding()

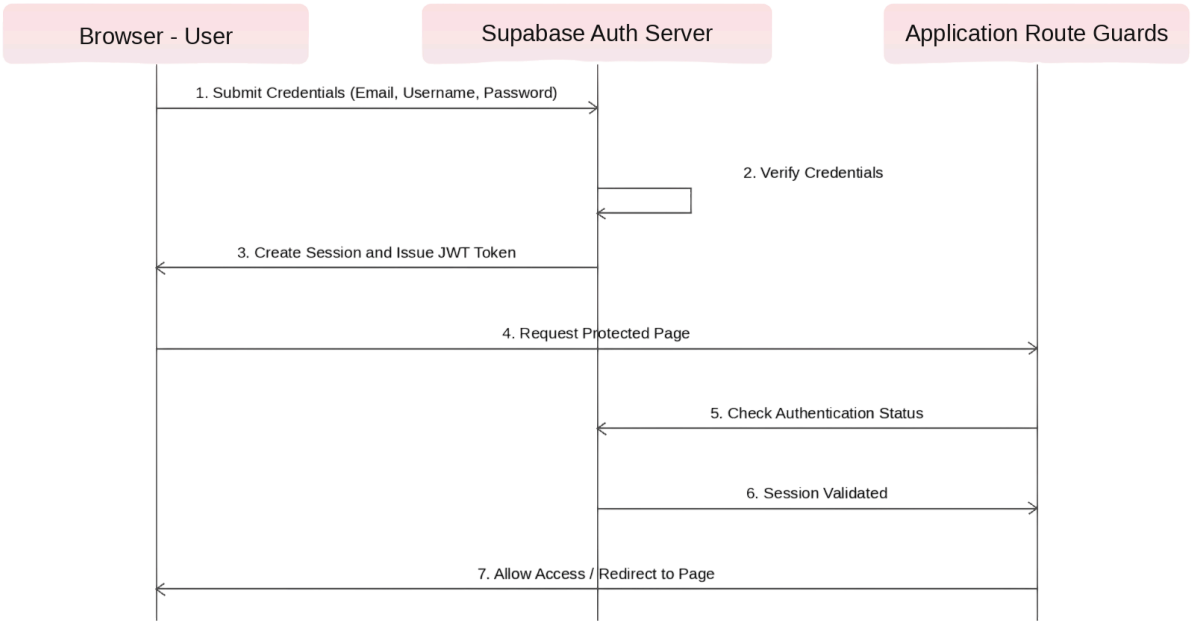
These functions serve as route-protection checks to verify whether the user has a valid Supabase session before allowing access to protected routes.

Route protection guards behaviour

Condition	*Behaviour*
Logged in	Can access protected routes
Not logged in	Redirected to `/login`
Logged in + not completed tutorial	Redirected to `/tutorial`
Logged in + not configured parameters	Redirected to `/parameters`
Logged in + no completed attempt	Cannot access `/summary` directly

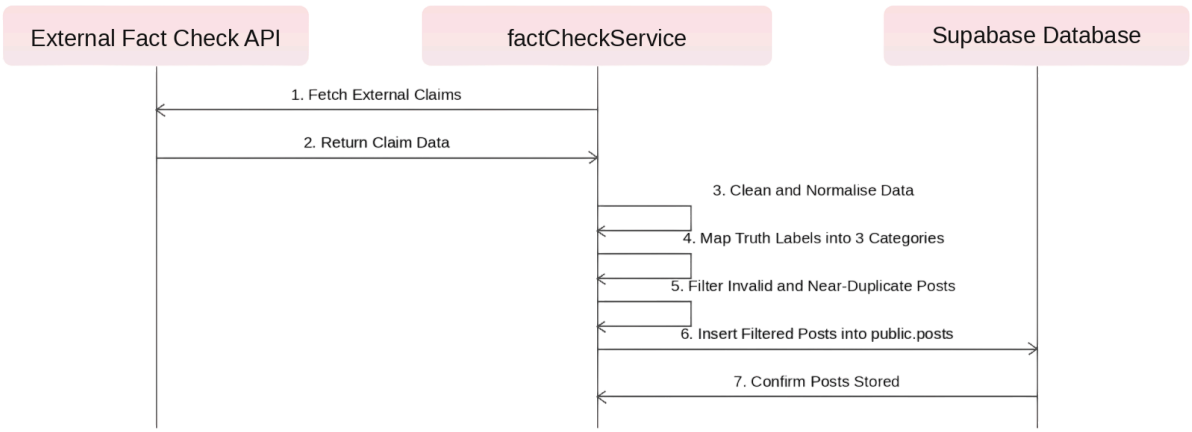
9.2 Login Operation

Flow



9.3 Posts Import Operation

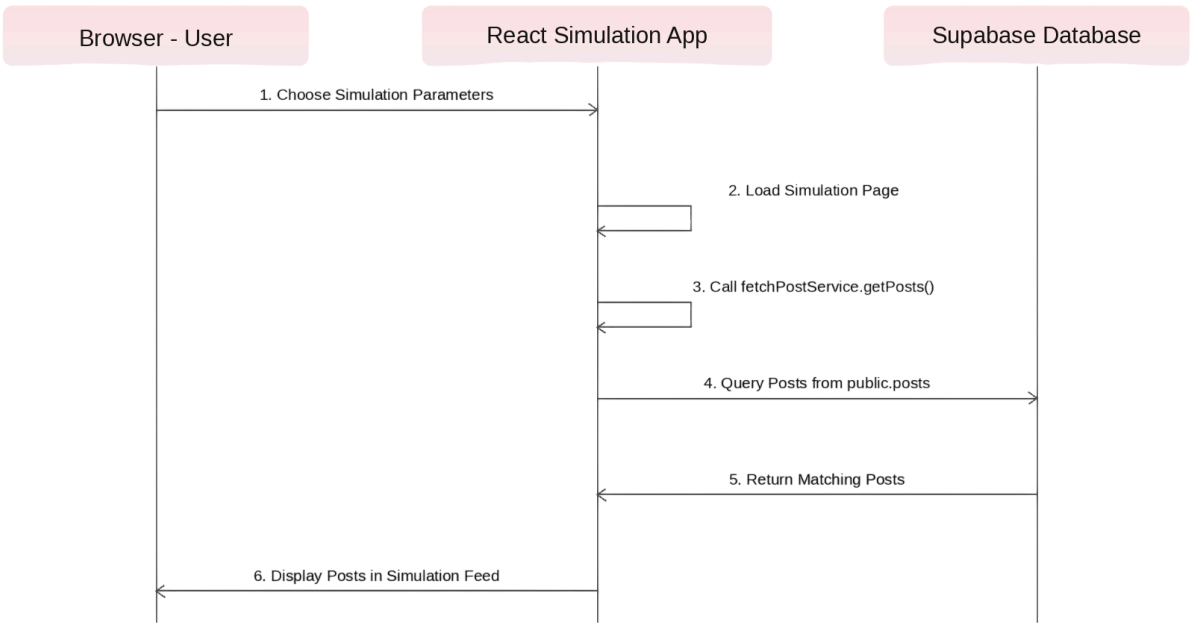
Flow



Rule	*Default Value*	*Behaviour*
`maxPerSubTopic`	6	Rejects a post candidate once its `sub_topic` has already reached this limit
`maxPerDomain`	4	Rejects once the post candidate's source URL hostname has reached this domain limit
`maxPerPublisher`	10	Rejects once the named publisher has reached this post limit
`similarityThreshold`	0.6	Rejects a post candidate as a near-duplicate if its Jaccard similarity to any already-accepted post in the same sub_topic is ≥ 0.6

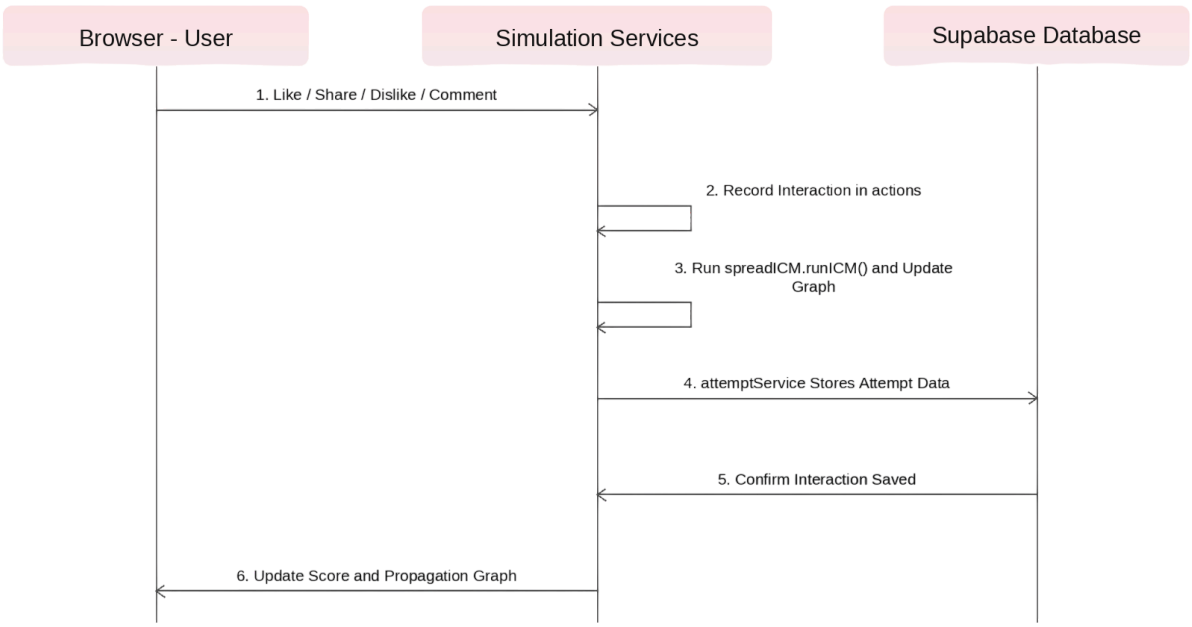
9.4 Fetch Posts Operation

Flow



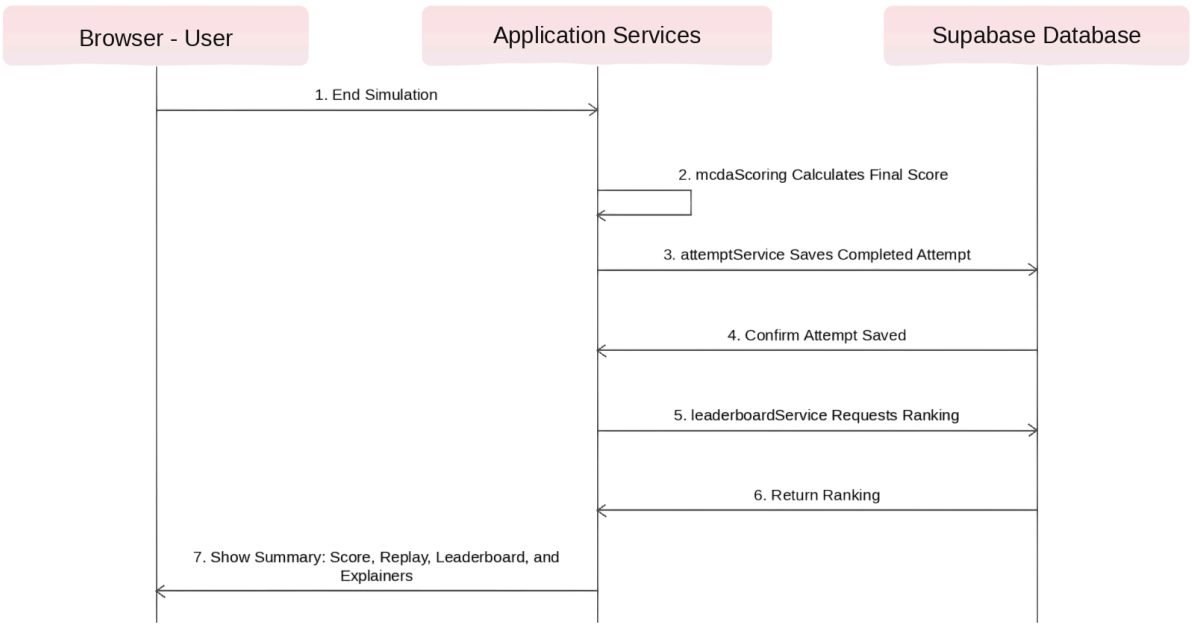
9.5 Save Interaction Operation

Flow



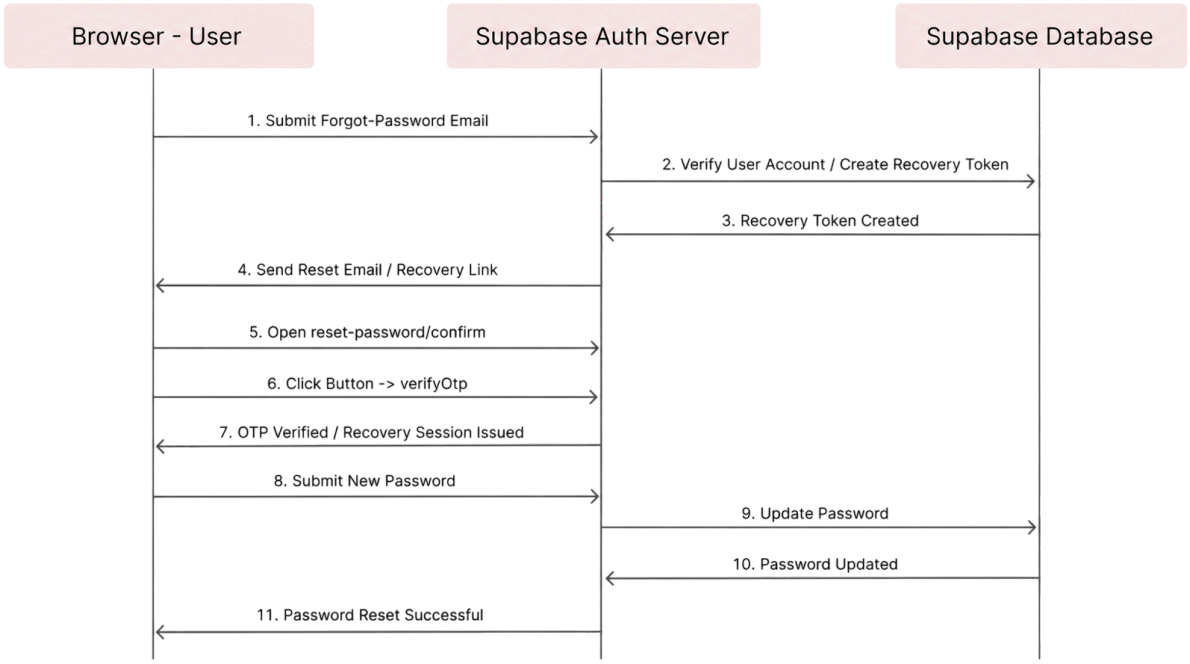
9.6 Save Attempt Operation

Flow



9.7 Password Reset Operation

Flow



10. Deployment

The web application is fully deployed and is accessible via URL: https://echo-echo-self.vercel.app/ .

Frontend & Serverless Hosting (**Vercel**): The application is deployed on Vercel. Vercel was chosen for its native support for the framework, and automated CI/CD pipeline. Every push to the production branch automatically triggers a new build deployment, while pull requests generate isolated preview deployments for testing.

Database Infrastructure (**Supabase**): The application's database layer is hosted on Supabase. This ensures high availability, secure connection pooling, and seamless integrations with our backend workflows.

Environment Configuration: All sensitive credentials, database connection strings, and third-party API keys are strictly decoupled from the codebase and managed via Vercel’s encrypted environment variable configuration.

11. Testing Strategy

This is designed to verify correctness across various layers, including but not limited to:

- Database
- Service
- Route protection
- ICM model
- MCDA scoring model
- Full user flow

Tests are divided into 4 main categories:

- Database validation tests
- Unit tests
- Integration tests
- User tests

This structure ensures that individual modules work correctly in isolation and that the complete application flow works correctly when everything comes together.

11.1 Database Validation Testing

This is carried out to ensure that data imported from Google Fact Check API is stored correctly in the Supabase PostgreSQL database.

The main table tested in this section is the *posts* table. The following tests ensure the following:

- expected columns exist
- imported truth labels are normalised correctly
- virality values stay within the correct range
- duplicate post content is not inserted
- publisher data is stored correctly
- non-english source URLs and contents are filtered out

These tests are very important because the quality of the simulation feed, scoring logic, and post explanations depend heavily on this specific table.

11.1.1 `Posts` table schema validation

This checks whether the posts table contains the expected columns and data types, ensuring the frontend and post retrieval service can safely access required fields such as `content`, `truth_label`, `topic`, `source_url`, and `virality`.

SQL test

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'posts';
```

Expected output

```
| column_name | data_type                |
| id          | uuid                     |
| publisher   | text                     |
| content     | text                     |
| picture_url | text                     |
| truth_label | text                     |
| topic       | text                     |
| explanation | text                     |
| created_at  | timestamp with time zone |
| source_url  | text                     |
| sub_topic   | text                     |
| virality    | real                     |
```

Pass criteria

Test passes if all required columns are present and each column has the expected data type.

11.1.2 Truth label normalisation validation

This checks that all imported fact-check ratings are normalised into only 3 labels: `False, Partly True, True`

SQL test

```
SELECT truth_label, COUNT(*)
FROM posts
GROUP BY truth_label;
```

Expected output

```
| truth_label | count |
| ----------- | ----- |
| False       | 213   |
| True        | 13    |
| Partly True | 36    |
```

Pass criteria

Test passes if no unexpected truth label appears in the result.

11.1.3 Virality Range Validation

This checks whether each post has a virality value within the expected range for this truth label.

Virality ranges are:

False	0.5 - 1.0
Partly True	0.3 - 0.7
True	0.1 - 0.5

SQL test (False)

```
SELECT *
FROM posts
WHERE truth_label = 'False'
AND (
    virality < 0.5
    OR virality > 1.0
);
```

Expected output

```
0 rows returned
```

SQL test (Partly True)

```
SELECT *
FROM posts
WHERE truth_label = 'Partly True'
AND (
    virality < 0.3
    OR virality > 0.7
);
```

Expected output

```
0 rows returned
```

SQL test (True)

```
SELECT *
FROM posts
WHERE truth_label = 'True'
AND (
    virality < 0.1
    OR virality > 0.5
);
```

Expected output

```
0 rows returned
```

Pass Criteria

Test passes if all three queries return 0 rows, meaning every post has a valid virality value.

11.1.4 Duplicate detection

This checks whether duplicate post content exists in the `posts` table. Duplicate posts are avoided because repeated posts may affect the realism of the simulation feed.

SQL test

```
SELECT
    content,
    COUNT(*)
FROM posts
GROUP BY content
HAVING COUNT(*) > 1;
```

Expected output

```
0 rows returned
```

Pass criteria

Test passes if no duplicate post content is found.

11.1.5 Publisher validation

This checks whether posts are imported from the expected fact-checking publishers.

SQL test

```
SELECT
    publisher,
    COUNT(*)
FROM posts
group by publisher;
```

Expected output

```
| publisher     | count |
| ------------- | ----- |
| PolitiFact    | 64    |
| Snopes        | 69    |
| USA Today     | 27    |
| FactCheck.org | 102   |
```

Pass criteria

Test passes if publisher names are stored correctly and match the expected imported dataset.

11.1.6 English content validation

This checks for common non-english URL patterns in source_url. This ensures that the posts shown in the simulation are suitable for English-speaking users.

SQL test

```
SELECT id, source_url
FROM posts
WHERE source_url ILIKE '%/id/%'
   OR source_url ILIKE '%/fr/%'
   OR source_url ILIKE '%/es/%'
   OR source_url ILIKE '%/pt/%'
   OR source_url ILIKE '%/ar/%';
```

Expected output

```
0 rows returned
```

Pass criteria

Test passes if no non-English URL patterns are found.

11.2 Unit Test

These tests are carried out using Vitest. They focus on individual functions, services, and route actions. Mocking was used where needed so that each unit could be tested without relying on the full application flow.

Tests cover:

- post retrieval service from Supabase
- saving user simulation attempts
- inserting attempts with logged-in user ID
- route protection helpers
- MCDA scoring calculation
- ICM spreading logic

11.2.1 Post retrieval service from Supabase

This validates the post retrieval service used to fetch posts from Supabase `posts` table and return them to the frontend.

Test file

`app/tests/fetchPostService.test.ts`

Test case	*What it checks*	*Expected result*
Successful post retrieval	Supabase returns post data	Returns an array of posts
Empty post table	Supabase returns an empty array	Returns an empty array
Database failure	Supabase returns an error	Throw database error

Technical approach

A mocked Supabase-like client is used to simulate different database responses. This allows the test to verify service behavior without requiring a real database call.

Pass criteria

```
✓ app/tests/fetchPostService.test.ts (3 tests) 2ms
  ✓ getPosts (3)
    ✓ returns posts successfully 1ms
    ✓ returns empty array when no posts exist 0ms
    ✓ throws database error 0ms

Test Files  1 passed (1)
     Tests  3 passed (3)
  Start at  14:53:40
  Duration  90ms (transform 9ms, setup 0ms, import 33ms, tests 2ms, environment 0ms)
```

11.2.2 Save attempt service

This validates the frontend service responsible for sending completed simulation attempts to the backend route.

Tested service sends a `POST` request to `/save-attempt`
Test file

`app/tests/attemptService.test.ts`

Test case	*What it checks*	*Expected result*
Successful attempt save	Completed attempt sent to `/save-attempt`	Return the saved attempt
Failed attempt save	Server returns an error response	Throw error message

Technical approach

Global `fetch` function is mocked using Vitest, allowing the test to check whether the service sends the correct request method, headers, and JSON body.

Pass criteria

```
✓ app/tests/attemptService.test.ts (2 tests) 12ms
  ✓ attemptService.saveAttempt (2)
    ✓ successfully pushes the completed attempt to save-attempt.tsx 10ms
    ✓ successfully throws server error message when save fails 1ms

Test Files  1 passed (1)
     Tests  2 passed (2)
  Start at  18:16:41
  Duration  80ms (transform 6ms, setup 0ms, import 11ms, tests 12ms, environment 0ms)
```

11.2.3 Save attempt route action

This validates the backend route action that inserts a completed simulation attempt into the Supabase `attempts` table.

Test file

`app/tests/save-attempt.test.ts`

Test case	*What it checks*	*Expected result*
Logged-in user saves attempt	Route reads current user ID and inserts attempt	Returns status `200` and saved attempt
No logged-in user	Route cannot find authenticated user	Returns status `401` and error message

Technical approach

Supabase server client is mocked. Test mocks:

```
supabase.auth.getUser()
```

```
supabase.from("attempts")
.insert()
.select()
.single()
```

Pass criteria

```
✓ app/tests/save-attempt.test.ts (2 tests) 12ms
  ✓ save-attempt route action (2)
    ✓ successfully inserts an attempt using the currently logged-in user's id 11ms
    ✓ successfully returns 401 when there is no logged-in user 0ms

Test Files  1 passed (1)
     Tests  2 passed (2)
  Start at  18:14:30
  Duration  102ms (transform 17ms, setup 0ms, import 24ms, tests 12ms, environment 0ms)
```

11.2.4 Route protection

This validates the route guard helpers used to protect certain pages and enforce the user flow.

Test file

`app/tests/auth.server.test.ts`

Functions tested

`requireLogin()`	Ensures user is authenticated
`requireParameter()`	Prevents direct access to `/simulation` without parameters
`requireAttempt()`	Prevents direct access to `/summary` without an attemptId
`requireOnboarding()`	Allows only first-time users to access the tutorial
`requireTutorial()`	Ensures user is logged in and has completed tutorial

Test cases for `requireLogin()` – 3 tests

Test case	*What it checks*	*Expected result*
Logged-out user opens protected route	No authenticated user exists	Redirect to `/login?redirectTo=...`
Supabase auth error	Auth session cannot be validated	Redirect to `/login?redirectTo=...`
Logged-in user	Auth session validated	Route is allowed

Test cases for `requireParameter()` – 9 tests

Test case	*What it checks*	*Expected result*
Logged-out user opens `/simulation`	User is not authenticated	Redirect to `/login`
First-time user opens `/simulation`	Onboarding is incomplete	Redirect to `/tutorial`
Returning user opens `/simulation` without parameters	Required query parameters are missing	Redirect to `/start`
Missing followers	followers is not in URL	Redirect to `/start`
Missing followings	followings is not in URL	Redirect to `/start`
Missing density	density is not in URL	Redirect to `/start`
Missing topics	topics is not in URL	Redirect to `/start`
All required parameters exist	URL contains required configuration	Route is allowed

Test cases for `requireAttempt()` – 4 tests

Test case	*What it checks*	*Expected result*
Logged-out user opens `/summary`	User is not authenticated	Redirect to `/login`
First-time user opens `/summary`	Onboarding is incomplete	Redirect to `/tutorial`
Returning user opens `/summary` without attemptId	attemptId is missing	Redirect to `/start`
Returning user opens `/summary` with attemptId	attemptId exists	Route is allowed

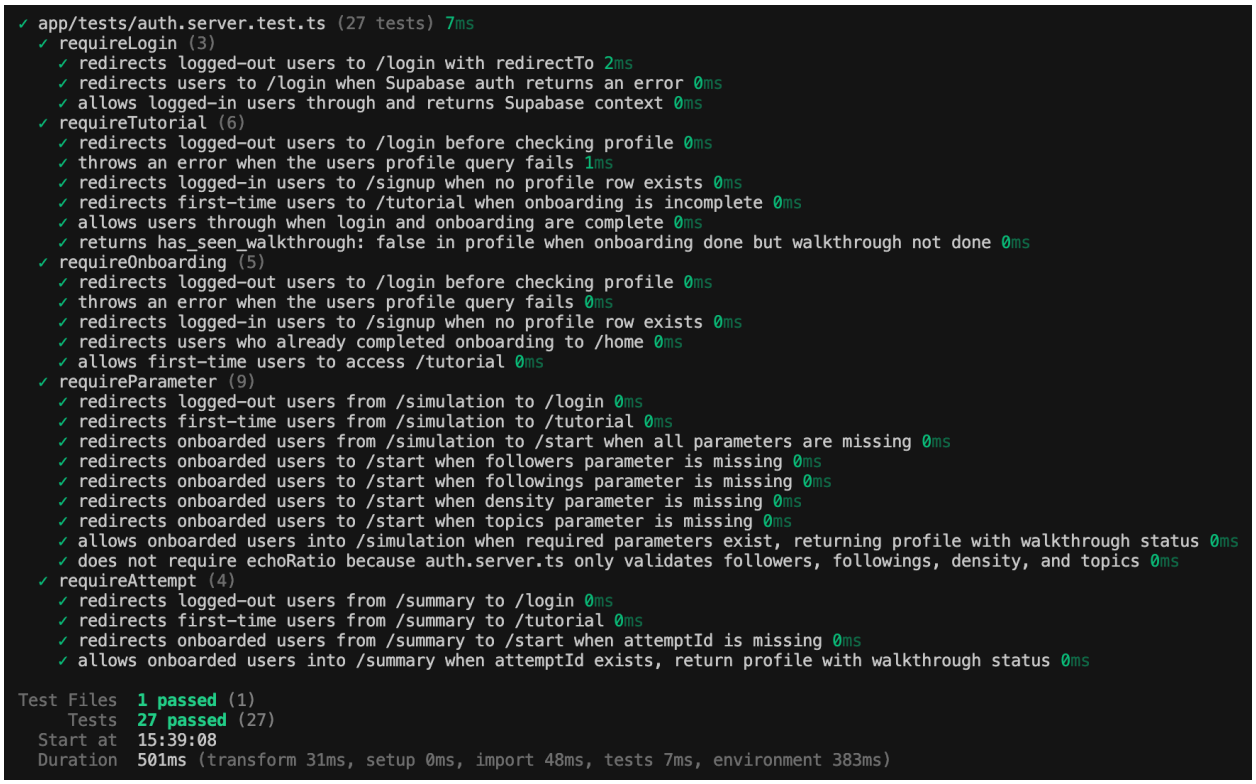
Test cases for requireOnboarding() – 5 tests

Test case	*What it checks*	*Expected result*
Logged-out user opens `/tutorial`	User is not authenticated	Redirect to `/login`
Profile query error	Supabase profile query fails	Throw error
No profile row	Auth user exists but no `users` row exists	Redirect to `/signup`
Returning user opens `/tutorial`	User already completed onboarding	Redirect to `/home`
First-time user opens `/tutorial`	User hasn't completed onboarding	Route is allowed

Test cases for `requireTutorial()` – 6 tests

Test case	*What it checks*	*Expected result*
Logged-out user	User is not authenticated	Redirect to `/login`
Profile query error	Supabase profile query fails	Throw error
No profile row	Auth user exists but no `users` row exists	Redirect to `/signup`
First-time user	`has_seen_onboarding = false`	Redirect to `/tutorial`
Returning user	`has_seen_onboarding = true`	Route is allowed
Walkthrough pending	Onboarding done, but `has_seen_walkthrough = false`	Route is allowed, returned `profile.has_seen_walkthrough = false`

Pass criteria



11.2.5 Filter posts

This validates the text-processing utilities responsible for parsing URL search parameters.

Test file

`app/tests/filter-posts.test.ts`

Test case	*What it checks*	*Expected result*
Missing parameter fallback	Checks how the utility handles null URL topic parameter	Returns an array containing [‘No Preference’]
Single topic	Checks if a single input is converted into a standard array format	The topic selected is ‘Society’. Returns an array [‘Society’]
Comma-separated parsing	Checks if multiple topics listed in the URL are split correctly	Topics selected are Society, Environment, Technology. Returns an array [‘Society’, ‘Environment’, ‘Technology’]
Whitespace trimming	Checks if accidental spaces around topic words are removed	Removes extra spaces, transforming " Society , Environment " into ["Society", "Environment"].
Empty input cleanup	Checks if trailing commas or duplicate commas are handled	Transform "Society,,Environment," to ["Society", "Environment"].

Technical approach

This test uses Vitest and is tested in isolation..

Pass criteria

```
✓ app/tests/filter-posts.test.ts (5 tests) 2ms
  ✓ handleTopics() Processing unit tests (5)
    ✓ should return ['No Preference'] when URL completely missing 1ms
    ✓ should process a single topic correctly into an array string 0ms
    ✓ should parse multiple topics split by commas 0ms
    ✓ should automatically trim accidental whitespaces around individual values 0ms
    ✓ should strip out empty indices caused by trailing or duplicate commas 0ms

 Test Files  1 passed (1)
      Tests  5 passed (5)
   Start at  17:21:38
   Duration  497ms (transform 14ms, setup 0ms, import 21ms, tests 2ms, environment 406ms)
```

11.2.5.1 Manual testing log

Beyond the unit tests, manual testing was performed directly inside the application browser environment to confirm that UI filters correctly map to the database outputs.

Manual testing log:
https://docs.google.com/document/d/13n1edg7BcwbGOIoZY3mcfQkDJbrMchSng2NEyYs_i_0/edit?usp=sharing

11.2.6 MCDA scoring

This validates the MCDA scoring functions used to calculate the user’s final score.

Tested functions:

```
calculateA()
```

```
calculateB()
calculateC()
calculateFinalScore()
calculateAllScores()
getScoreLabel()
```

Test file

```
app/tests/mcdaScoring.test.ts
```

Scoring components tested

A	Weighted accuracy score
B	Containment score
C	Learning engagement score
Final score	Average of A, B, and C on a 100-point scale

Test case	*What it checks*	*Expected result*
Empty action list	No user actions exist	Accuracy score = 0
Standard interactions	Weighted average is calculated correctly	Return expected weighted score
Negative impact score	Score is below zero before clamping	Score = 0
Zero / negative network size	Invalid network size	Containment score = 0
Zero infected nodes	No spread occurred	Containment score = 1
Partial infection	Some nodes were infected	Return correct containment ratio
Over-infection boundary	Infected nodes exceed network size	Score = 0
Zero explainers opened	No explainers exist	Learning engagement score = 0
Partial explainer usage	Some explainers were opened	Return correct learning ratio
Final score calculation	A, B, C are averaged	Score converted to base 100
Perfect profile	Best possible performance	Final score = 100
Low-performing profile	Bad action, high infection, no learning	Return low final score

Score input	*Expected label*
0 - 29	😈 The Supervillain
30 - 44	😖 Careless Rookie
45 - 59	🙈 Neutral NPC
60 - 74	🔍 Rookie Detective
75 - 89	😇 Guardian Angel
90 - 100	🔥 The Super Hero
150 (above range)	🔥 The Super Hero (clamped down)
-20 (below range)	😈 The Supervillain (clamped up)

Technical approach

This mocks `impactQualityScores` (from `attemptLog`) to four fixed values, good = 1.0, neutral = 0.5, bad = 0.0, fail = -0.5, so scoring behavior is predictable and isolated from the real impact-quality mapping. The fail value is specifically used to test negative-score clamping.

Pass criteria

```
✓ app/tests/mcdaScoring.test.ts (26 tests) 3ms
  ✓ MCDA Scoring Formula (26)
    ✓ calculateA: Weighted Accuracy Score (3)
      ✓ should return 0 if the actions array is empty 1ms
      ✓ should calculate correct weighted average for standard interactions 0ms
      ✓ should safely clamp outcomes to 0.0 lower bound if impact scores are negative 0ms
    ✓ calculateB: Containment Score (4)
      ✓ should return 0 if the network size is evaluated as zero or negative 0ms
      ✓ should return 1.0 if zero nodes were infected (perfect containment) 0ms
      ✓ should calculate proportion accurately 0ms
      ✓ should yield 0 if the infection completely overruns the network population boundary 0ms
    ✓ calculateC: Learning Engagement Score (2)
      ✓ should return 0 safely if total explainers is zero 0ms
      ✓ should return a ratio proportional to total number of explainers 0ms
    ✓ calculateFinalScore (1)
      ✓ should evaluate balanced profiles up to a 100 base scale 0ms
    ✓ calculateAllScores (2)
      ✓ should evaluate a perfect user profile with maximum points across metrics 0ms
      ✓ should evaluate a low-performing scenario correctly 0ms
    ✓ getScoreLabel: Score Labels (14)
      ✓ should label 0 as The Supervillain (lower bound) 0ms
      ✓ should label 29 as The Supervillain (upper bound) 0ms
      ✓ should label 30 as Careless Rookie (lower bound) 0ms
      ✓ should label 44 as Careless Rookie (upper bound) 0ms
      ✓ should label 45 as Neutral NPC (lower bound) 0ms
      ✓ should label 59 as Neutral NPC (upper bound) 0ms
      ✓ should label 60 as Rookie Detective (lower bound) 0ms
      ✓ should label 74 as Rookie Detective (upper bound) 0ms
      ✓ should label 75 as Guardian Angel (lower bound) 0ms
      ✓ should label 89 as Guardian Angel (upper bound) 0ms
      ✓ should label 90 as The Super Hero (lower bound) 0ms
      ✓ should label 100 as The Super Hero (upper bound) 0ms
      ✓ should clamp scores above 100 down to The Super Hero 0ms
      ✓ should clamp negative scores up to The Supervillain 0ms

Test Files  1 passed (1)
     Tests  26 passed (26)
  Start at  15:46:48
  Duration  290ms (transform 21ms, setup 0ms, import 31ms, tests 3ms, environment 201ms)
```

11.2.7 ICM spreading logic

This validates the execution of the Independent Cascade Model spreading logic.

Tested functions:

```
calculateActivationProbability()
runICM()
```

Test file

`app/tests/icm-model.test.ts`

Test case	*What it checks*	*Expected result*
User action scaling	Checks that user interactions scale the cascade probability correctly according to the algorithmic weights (A)	Share > Comment > Like > Ignore > Dislike hierarchy is maintained
Truth label modifiers	Checks that a post's fact-checked truth_label applies the correct modifiers to edge activation	False content spreads faster than factual content, verify the constraint: False > Partly True > True
Probability boundary cap	Checks the mathematical clipping threshold	Total computed probability values are strictly upper-bounded at 1.0
Sequential wave cascade	Check the fundamental rule of discrete ticks. Tracks chronological propagation across network step-by-step	Simulation progresses in sequence across the discrete tick, activating wave-by-wave
Single-chance activation	Checks the fundamental rule of single-chance activation. Prevent multiple concurrent activations.	When a node is exposed to simultaneous spreading from multiple neighbours, it should only be activated once and ignore other attempts

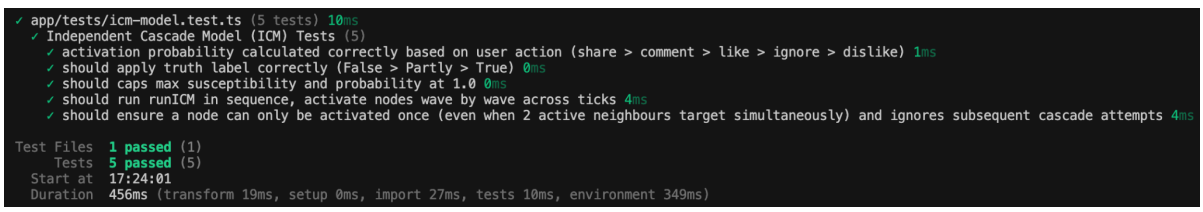
Technical approach

This test uses Vitest to run isolated, predictable tests against the core spreading logic without external database dependencies.

This test uses static mock payloads (SimulatedPost objects), representing extreme cases: ranging from highly infectious, fake content to highly verified real news.

Network environments are also mocked using explicit relational Map schemas (Map<string, SimulatedNode>).

Pass criteria



11.3 Integration Test

11.3.1 User flow and route guards

This checks that the main pages, route guards, Supabase authentication, onboarding status, parameter gate, simulation flow, and attempt saving work together correctly

Test file

`app/tests/user-flow.e2e.test.ts`

Tools used:

- Playwright: opens the app in a browser and performs real user actions
- Supabase Admin Client: creates and removes test users safely
- Dotenv: loads local test environment variables
- React Router loaders/actions: tested through real route navigation

Test case	*What it checks*	*Expected result*
Protected routes	Logged-out users visit protected pages directly	Redirected to `/login?redirectTo=...`
First-time signup	New user signs up through the Ul	Redirected to `/tutorial`
Tutorial gate	User tries to finish tutorial before all actions	Finish button stays disabled
Tutorial completion	User completes one action for every tutorial post	Tutorial can be submitted
Tutorial locked after completion	Completed user opens `/tutorial` again	Redirected to /
Simulation parameter gate	User opens `/simulation` without parameters	Redirected to `/start`
Summary attempt gate	User opens `/summary` without attempt ID	Redirected to `/start`
Main returning-user flow	Login → Start → Parameters → Simulation → Summary	Attempt is saved, direct to `/summary`

Test flow

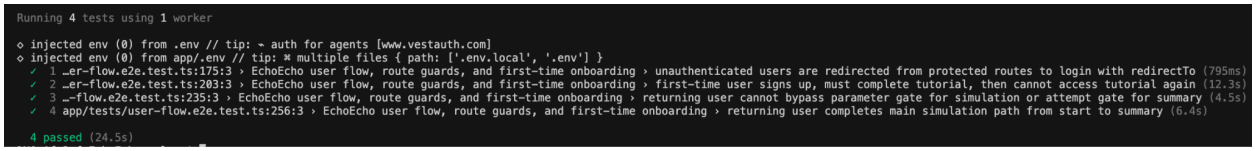
1. Open protected pages while logged out
2. Confirm each protected page redirects to `/login?redirectTo=...`
3. Create a first-time user through the real signup page
4. Confirm the user is redirected to the onboarding tutorial
5. Check that the tutorial cannot be completed before all tutorial posts are interacted with
6. Complete one action for each tutorial post
7. Submit the tutorial
8. Confirm the user is redirected to the authenticated `/home` page

9. Confirm the user cannot access `/tutorial` again after onboarding is completed
10. Create a returning user using the Supabase Admin Client
11. Log in through the real `/login` page
12. Attempt to access `/simulation` without parameters
13. Confirm redirection to `/start`
14. Attempt to access `/summary` without an `attemptId`
15. Confirm redirection to `/start`
16. Complete the returning-user flow from `/home` to `/summary`
17. Confirm the `/summary` page appears with an `attemptId`

Run command

```
npm run dev
npx playwright test "app/tests/user-flow.e2e.test.ts"
```

Pass criteria



11.3.2 Simulation UI + ICM + MCDA

This integration test validates that the Simulation page UI workflow bridges user interaction with the backend ICM spread logic and MCDA scoring.

Test file

`app/tests/simulation.integration.test.tsx`

Test case	*What it checks*	*Expected result*
Database loaded	Checks if posts are fetched from database when the page loads	Test should find post text on screen
Clicking engagement buttons (eg. share) triggers simulation	Checks if clicking Share button actually triggers the ICM spread logic	`runICM()` should be called with the correct post and action data
Duplicate click guard	Checks that clicking Like multiple times does not save duplicate actions	Only one Like action is saved in `actionLog` when simulation ends
Comment modal selection	Checks if selecting a specific comment options saves the correct comment type	Chosen comment text maps correctly to the correct comment ID
Saving final MCDA score	Checks if click End Simulation saves all final score data	`saveAttempt()` should be called with the exact scores calculated at the end of simulation

Technical approach

This test uses Vitest and React Testing Library environment.

This test uses createMemoryRouter, together with custom parameterised URLs (`/simulation?followers=3&followings=0&density=Sparse&echoRatio=0&topics=Health`). This allows the tests to replicate actual application flow, ensuring filters from previous setup propagate down smoothly.

This test uses `vi.mock('cytoscape')` to mock the real cytoscape canvas with JavaScript object. This safely catches all the drawing, layout, and styling commands.

This test uses Vitest's spy feature (`vi.spyOn`) to watch how data moves between frontend UI and backend services.

Pass criteria

```
✓ app/tests/simulation.integration.test.tsx (5 tests) 214ms
  ✓ Simulation Integration (5)
    ✓ loads posts from database 39ms
    ✓ clicking share calls runICM() 52ms
    ✓ clicking like twice only records one action 40ms
    ✓ comment flow stores selected comment type 55ms
    ✓ End Simulation saves calculated MCDA score 28ms

Test Files  1 passed (1)
     Tests  5 passed (5)
  Start at  15:52:39
  Duration  824ms (transform 64ms, setup 0ms, import 180ms, tests 214ms, environment 360ms)
```

11.3.2.1 Integration Bug and Fixes

3 major bugs were faced during the integration testing of the simulation interface. Below is the documentation of them:

1. Cytoscape canvas routing crash
    a. Problem: Integration test suite would crash with errors saying could not find visual HTML structures.
    b. Cause: Cytoscape.js requires a physical browser window to calculate where to draw the network graph. As the terminal test environment run by Vitest has no visual screen, cytoscape crashed.
    c. Fix: Did a mock setup using vi.mock('cytoscape').
2. Score state mismatch when ending simulation
    a. Problem: The test case checking if scores saved correctly failed because data sent to saveAttempt was blank or not in sync with what was on screen.
    b. Cause: When the user clicks End Simulation, the test checks database payload immediately before the frontend state has finished calculating the final score.
    c. Fix: Added await waitFor(). This forces Vitest to wait until the asynchronous React state finishes updating and passes the final score down to saveAttempt before checking it.
3. Stale state data when clicking End Simulation
    a. Problem: When clicking End Simulation, the app would save empty arrays or outdated scores to database
    b. Cause: In simulation.tsx, React state updates (like setInfectedNodes) are asynchronous. When the user End Simulation, the handler function reads the state variables immediately. However, the function was trapped reading the old state, missing the latest updates.
    c. Fix: Added useRef hooks alongside tracking useEffect hooks for data (actionLogs, infectedNodes, and openedExplainerIds). Because refs mutate instantly and point directly to the same place in memory, updating ref inside useEffect enables endSimulation() to read the most updated data.

11.4 User Test

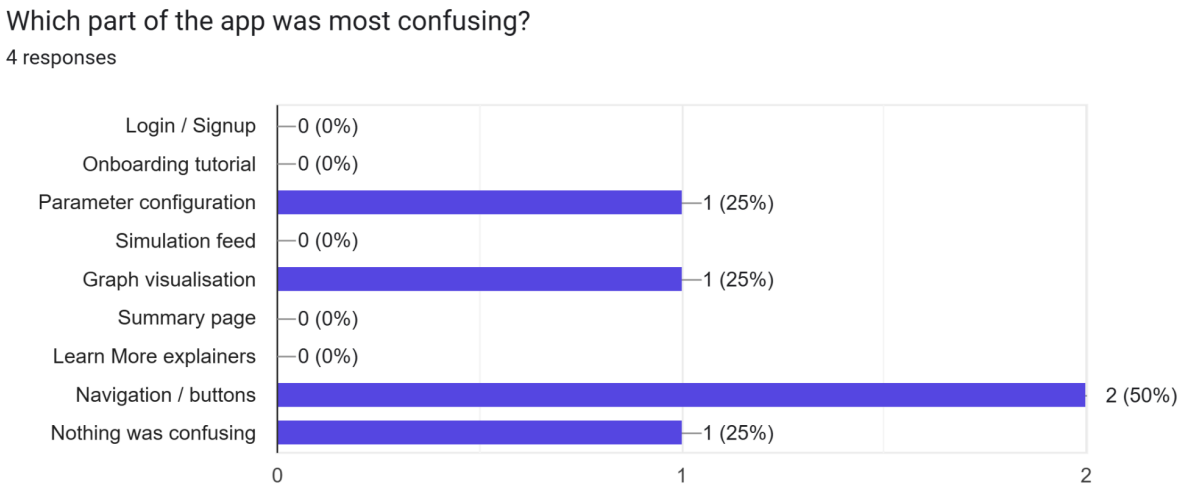
This test is conducted to evaluate whether users understood the purpose and operations of this app, whether the UI was easy to use, and whether the graph effects were clear.

Overall, results showed that users understood the overall purpose of the app, but **graph visualisation** and **app navigation / UI** were the most confusing areas.

Major findings:

11.4.1 Graph visualisation

From our user testing and peer evaluation feedback, one distinct problem reflected was that graph visualisation became cluttered when many nodes appeared. Users also reflect graph visualisation as one of the most confusing parts of our app.



Graph visual clarity:

Problem - Graphs gets cluttered very quickly when increasing the number of nodes, there is also lesser visual clarity between followers and following nodes. Not too sure what the tiny red hexagon next to the node meant either

Implemented, refer to Section 15.10

11.4.2 Nodes on graph

From our user testing, users wanted clearer distinction between followers, following nodes, and activated nodes.

2. Give more distinct and stronger colors to the following and follower nodes, so it can be easily differentiated when the number of nodes increase. Add a legend at the sidebar to indicate which color represents which node to reduce visual clarity

Implemented, refer to Section 15.10

11.4.3 Share button

From our user testing, users noticed that Share button could be repeatedly clicked on the same post.

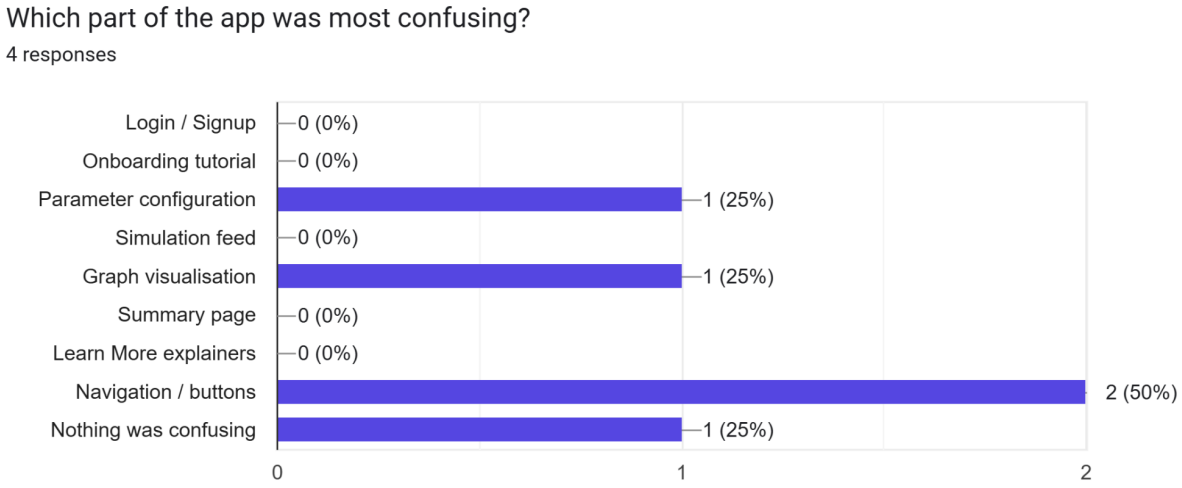
Problem - There is no clear explanation what each button(like, share, dislike, comment) does

Suggestion:
1. This is partly solved by having graph visual clarity but it can help more with user interactivity for them to see the immediate positive/negative impacts of each action, rather than a black box where they input a bunch of buttons and somehow output some values
2. It would also probably help to limit the number of actions a user can do for each piece activity feed, and maybe grey out the feed box once you hit the limit to indicate completion.
3. Also I dont know if this is an oversight or intended but I can spam share on a single activity feed and keep corrupting further nodes

Implemented, refer to Section 15.9

11.4.4 Explanation

From our user testing, users reflect that one of the most confusing part of the app was parameter configuration. A solution to this would be to provide contextual explanations to each factor.



Implemented, refer to Section 15.3

11.4.5 True vs false distinction

From our user testing, users suggested making true and false information spread distinguishable in onboarding tutorials while only not making it obvious during real simulation.

> 4. Another suggestion is to differentiate correct information and spread of misinformation by marking different colors. So for example by using the colored arrows and borders, if the user spread correct information, the arrows and borders gets marked green, misinformation marks red. This also feeds your users with dopamine which lets them stay longer on your website
> 5. If you want to keep that "unknown" factor of not knowing the consequences of your actions, you can first introduce them to the practice colored versions, then suggest them to the testing uncolored version once they are ready. Here the consequences will only show at the end and the colored version can be seen during the propagation playback.

Implemented, refer to Section 15.4

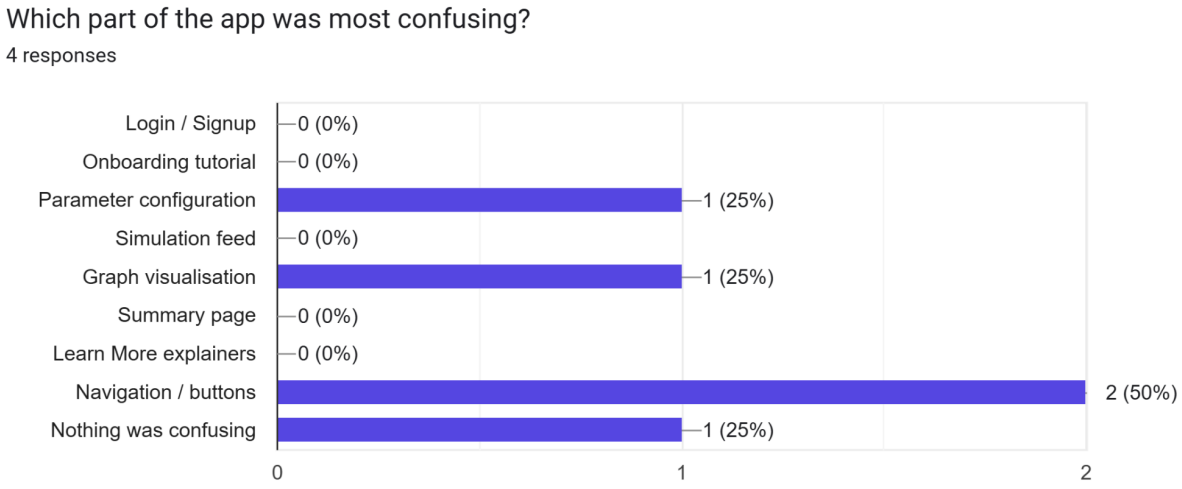
11.4.6 Password reset

During our self-evaluation, we realise that when User uses work/school email to sign up, and forget the password, the reset link does not work.

Implemented, refer to Section 15.2

11.4.7 Navigation buttons (i.e. logout, back buttons)

From our user testing, users reflect that the most confusing part of the app was navigation/ buttons.



Implemented, refer to Section 15.5 and 15.6

For the complete list of improvements implemented in Milestone 3, refer to Section 15.

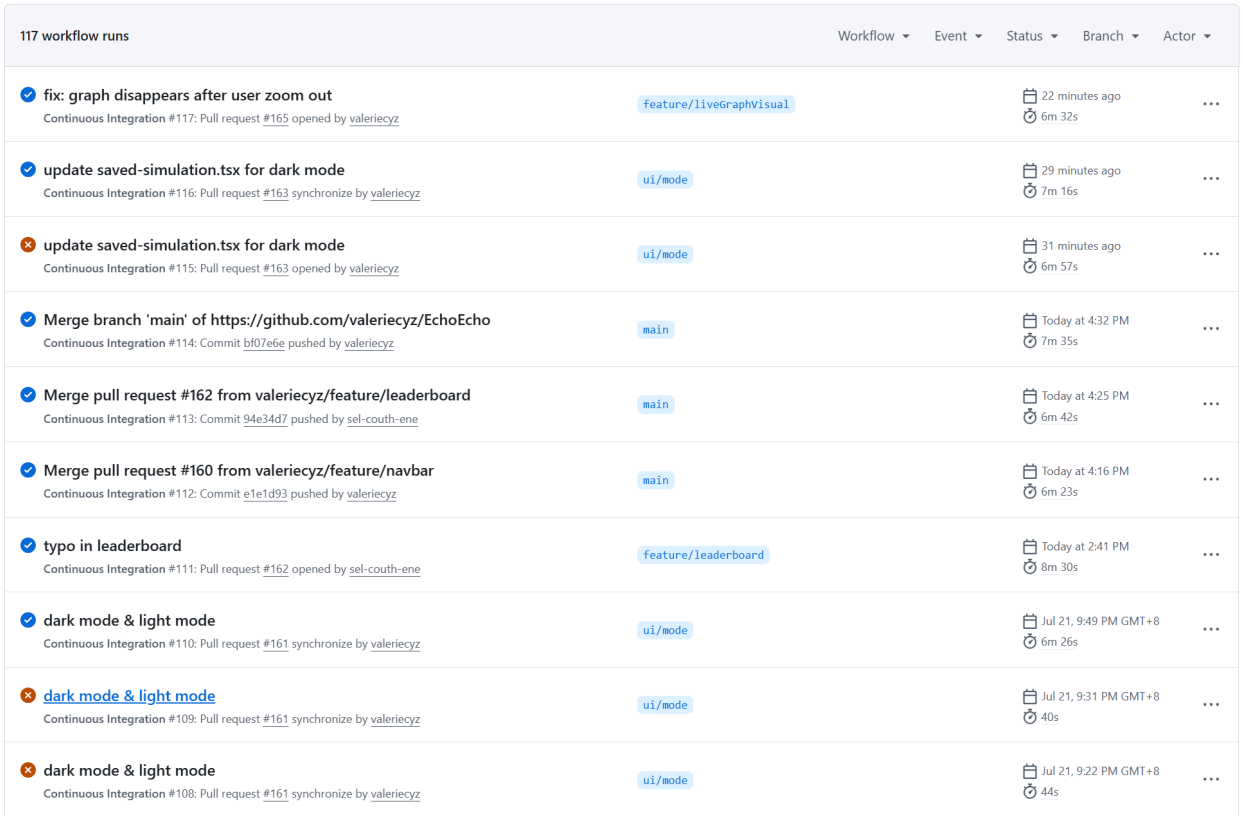
For the complete user testing response, refer to Section 17.4.

12. Continuous Integration/ Continuous Deployment (CI/ CD)

By combining automated deployments with Vercel for hosting and environment management and automated workflows with GitHub Actions, the development workflow has strict quality gates from initial pull requests to the final production deployment.

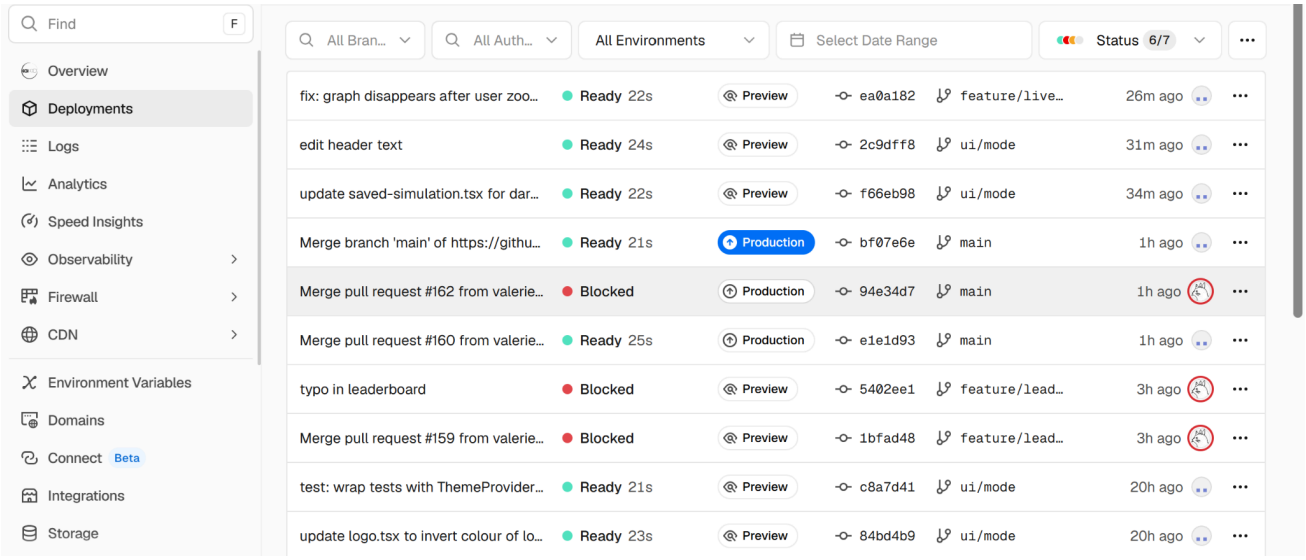
12.1 CI – GitHub Actions

CI is powered by GitHub Actions through automated workflows configured to execute on every push and pull requests to the main branch. This workflow act as a quality control gate, ensuring that the proposed changes conform to our repository standards before merging to main branch.



12.2 CD – Vercel

CD is done through integration with Vercel. Every pull request automatically triggers an isolated preview deployment, where a unique preview URL is generated, allowing developers to conduct manual and visual testing before code reaches production. Once a pull request passes all CI checks and is merged to the main branch, Vercel triggers a production build.


Find F
Overview
Deployments
Logs
Analytics
Speed Insights
Observability
Firewall
CDN
Environment Variables
Domains
Connect Beta
Integrations
Storage
All Bran...
All Auth...
All Environments
Select Date Range
Status 6/7
fix: graph disappears after user zoo... Ready 22s Preview ea0a182 feature/live... 26m ago
edit header text Ready 24s Preview 2c9dff8 ui/mode 31m ago
update saved-simulation.tsx for dar... Ready 22s Preview f66eb98 ui/mode 34m ago
Merge branch 'main' of https://githu... Ready 21s Production bf07e6e main 1h ago
Merge pull request #162 from valerie... Blocked Production 94e34d7 main 1h ago
Merge pull request #160 from valerie... Ready 25s Production e1e1d93 main 1h ago
typo in leaderboard Blocked Preview 5402ee1 feature/lead... 3h ago
Merge pull request #159 from valerie... Blocked Preview 1bfad48 feature/lead... 3h ago
test: wrap tests with ThemeProvider... Ready 21s Preview c8a7d41 ui/mode 20h ago
update logo.tsx to invert colour of lo... Ready 23s Preview 84bd4b9 ui/mode 20h ago

13. Security Considerations

13.1 Supabase Authentication

User authentication is implemented to manage user sign up and log in procedures. After successful login, Supabase creates a user session and provides a JWT token, which is used to identify the authenticated user when they access protected routes or perform certain actions.

13.2 Password Hashing

User passwords in authentication credentials are managed by Supabase Authentication while application specific user information and data are stored in the `public.users` table.

13.3 Row Level Security (RLS)

Having Row Level Security ensures that users can only access or modify specific database rows. This helps protect user privacy and data.

Table	*RLS behaviour*
`public.users`	Users can only view and update their own records
`public.attempts`	Users can only insert, view, and update their own simulation attempts
`public.posts`	Authenticated users can only read posts, but cannot modify post data

13.4 Input Validation

Validation implemented:

Sign up & log in	Username, email, password must not be empty
Parameter configuration	Follower count, following count, network density, echo chamber strength must be within valid ranges
Topic selection	At least one topic must be selected before proceeding to simulation
Post interactions	Action type must be valid (like / share / dislike / comment)
Post import	Imported posts should have english content, own topic and sub-topic, truth label etc.

13.5 Password Reset Link Security

Referring to Section 11.4 User Test Findings 11.4.6, we have discovered the following.

Corporate and ISP email security systems automatically prefetch every link in an incoming email to scan for phishing. Since Supabase recovery tokens are single-use, this automated prefetch was silently consuming the token, causing the real user's click to fail.

Therefore, we have made the following changes: the email link sent by Supabase does not directly trigger session creation, instead, it routes to an intermediate `/reset-password/confirm` page where loader only reads the token from the URL and renders a confirmation button. The actual call to `supabase.auth.verifyOtp()` that consumes the single-use recovery token and establishes a session, is only triggered when the user clicks the confirmation button.

If `verifyOtp()` fails regardless, such as when the link genuinely expires or was already used, the user is shown a clear message directing them to request a new reset link.

14. Maintenance & Troubleshooting

14.1 Known Issues

1. *Cytoscape Graph Layout*

The graph visualiser (Cytoscape.js) is highly optimised for desktop layout. On smaller mobile screens, dense network configurations can look very cluttered, making it difficult to read and understand.

Solution:

- Use different layouts for desktop and mobile, ie. add a simplified mobile graph layout
- Limit maximum graph density on mobile

2. *Fact Checking API*

The web app depends on data fetched and normalised from Google Fact Check Tools API. If Google modifies its API structure or alters its truth rating labels, the automated import classifier may require manual data mapping updates.

Possible improvements include:

- Regular review of the API response structure
- Update of the import script field mapping

14.2 Maintenance

1. *Database Cleaning*

Run seasonal optimization scripts to archive old simulation logs, so as to keep the system lightweight and highly response.

Actions to be taken:

- Archive old simulation attempts after a fixed period (eg. a year)
- Remove incomplete or failed attempts

15. Milestone 3 Enhancements

With reference to the Section 10.4 User Test, these are enhancements implemented in milestone 3.

15.1 Image Classifier Service Expansion

Previous Limitation

- too few general images for posts
- posts may reuse the same image
- sub-topics do not have a suitable image

Current Implementation

- built `imageService.ts` that queries the Pexels API with a text query and falls back to one of the 4 bundled stock images if the API key is missing or no result is returned
- tested with `imageService.test.ts`

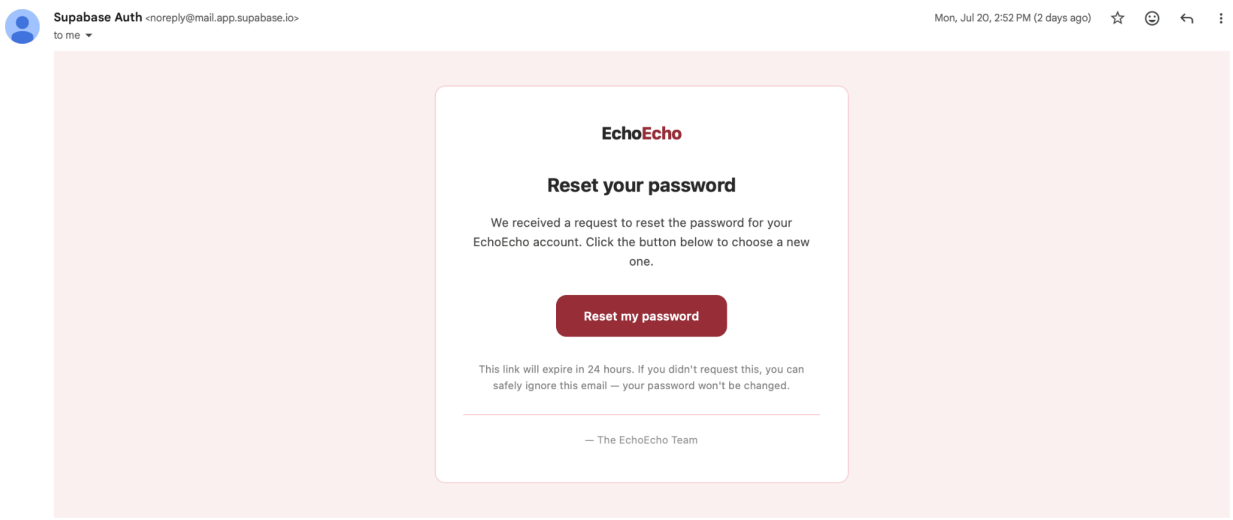
15.2 Forgot Password Flow During User Authentication

Previous Limitation

- login page does not have a forgot password option
- users cannot reset their password

Current Improvement

- `/forgot-password` collects an email and calls `supabase.auth.resetPasswordForEmail()`
- the reset link routes to the `/reset-password/confirm` page, deferring the actual `verifyOtp()` call to a user-initiated button click
- only after confirmation, the user is redirected to `/reset-password` to set a new password



15.3 Contextual Help Icons On Parameter Configuration

Previous Limitation

- terms on the parameters configuration page are hard to understand

Current Improvement

- added a `Tooltip` component and a `HELP_TEXT` map, wired to Followers Count, Followings Count, Network Density, Echo Chamber Ratio, and Topic Selection
- each icon shows a short explanation on click

15.4 Clearer Onboarding Tutorial

Previous Limitation

- can give more guidance
- some may still not fully understand how their actions affect the graph

Current Improvement

- each action now has clearer explanatory copy and feedback after it's taken
- "Redo Tutorial" entry was added to the global navigation menu, opening `/tutorial-replay`
- activationType tracking with distinct node colours per activation type was added to the graph, alongside a one-line description per replay tick

15.5 Back Button Improvements

Previous Limitation

- back buttons were inconsistent
- some pages do not clearly show where the user can go back to

Current Improvement

- build a reusable BackButton component used app-wide
- it calls the browser's native back navigation when history exists, and only falls back to a fixed route when there is no history to return to

15.6 Sign Out Behaviour Improvements

Previous Limitation

- sign out currently sends users to the login page
- some pages do not have a sign out button

Current Improvement

- sign out now opens a confirmation dialog before acting, then sends users to the intro page
- sign out buttons were rolled out to `/home`, `/start`, `/leaderboard`, `/learn-more`, `/saved-simulations`

15.7 Quit Simulation Without Saving Attempt

Previous Limitation

- simulation page only has an end simulation option, without quit button
- users cannot quit midway

Current Improvement

- add a separate `Stop Simulation` button with a confirmation modal
- user leaves the simulation without saving the attempt
- send user back to the parameter page

15.8 Learn More Explainer Cleanup

Previous Limitation

- some learn-more explainers may contain placeholder text
- some sections may show unnecessary dots or unclear empty content

Current Improvement

- Add a clear "No data entered in database row." message shown in place of a blank explanation

15.9 Cleaner Simulation Feed

Previous Limitation

- share button can be clicked repeatedly for the same post

Current Improvement

- each action button now disables itself and shows a checked state once used
- like and dislike are mutually exclusive
- share and comment are each single-use per post
- the whole card gets a highlighted tint once any action has been taken on it

15.10 Clearer Graph Visualisation

Previous Limitation

- nodes are not clearly visible and network looks cluttered, especially when user chooses Dense Network Density
- follower and following nodes are confusing and not easily distinguishable
- action buttons effect on the graph is confusing
- graph does not clearly explain what each visual element means

Current Improvement

- reduced default node size to 10px
- colour-coded follower vs following nodes and edges via a dedicated colour theme palette
- added a collapsible graph legend covering inactive/activated nodes and edge states

15.11 Interface Polish

Previous Limitation

- small UI issues exist
- some routes, buttons, loading states, or messages need cleanup

Current Improvement

- cleaned up route redirects, loading states, error states
- added a dark/light model theming
- made the graph and feed panels resizable via a drag handle
- fixed the graph-disappears-on-zoom-out bug
- logo click now returns to `/home`

15.12 Post Import Diversity and Feed Variation

Previous Limitation

- posts within the same topic/sub-topic were repetitive
- some shared images or came from near-identical claims

Current Improvement

- `postDiversityGate.ts` caps posts per sub-topic, per source domain, per publisher
- near-duplicate claims are rejected via Jaccard similarity over 3-word shingles at a 0.6 threshold

16. Potential Future Enhancements

16.1 AI-Powered Content Curation Pipeline

Current Limitation:
Currently, post content, truth labels, explainers, and fact-check links rely on fetching and normalising static data from the Google Fact-Check API. This limits the diversity, recency, and format of content presented in the simulated feed, as it restricts posts to claims that have already been classified by our external third party fact-checkers.

Proposed Enhancement:
Implement an AI ingestion and fact-checking pipeline using an LLM equipped with web access capabilities. The system would autonomously scan, fact-check, and curate real-world social media posts into our simulation database (`public.posts`).

- **Autonomous fact checking of claims:** The AI evaluates posts' content against reliable online sources, assigning a confidence score and determining the post's truth-label (True, Partly True or False).
- **Automated data normalisation:** Content is automatically transformed into EchoEcho's schema, generating standardised claim summaries, learn-more explainers, virality factors, and original source links.

16.2 Interactive "What-If" Counterfactual Learning

Current Limitation:
Currently, the tick-by-tick playback feature only offers a passive visual playback of actions recorded in spreadLog. Users can observe how information propagated sequentially based on their completed simulation session, but they cannot experiment with alternative decisions or test "what-if" scenarios on the existing network simulation.

Proposed Enhancement:
Implement a branching simulation engine that allows users to fork their simulation history directly from any point on the playback timeline.

- **Forking simulation history:** At the playback interface, users can pause at a specific point and select a counterfactual action. For eg. At tick 3, user initially shared post 1. Now, user can choose another action (eg, dislike or comment) to see how different action will result in different spread, under the same existing network conditions.
- **Branch simulation engine & dual graph rendering:** The spread logic re-simulates propagation from that tick onward under the new action done. Users can toggle between the actual path and the counterfactual path to compare side-by-side.
- **Comparative impact metrics:** Display clear change in score and network (eg. Infection rate: 85% vs 15%, Critical Thinker Score improved by 15), reinforcing the direct consequences of individual choices when interacting online.

16.3 Downloadable Summary Report

Current Limitation:
Currently, the post-simulation evaluation is contained entirely within the web application on the summary and saved-simulation pages. Users cannot export their simulation data, network visualisation, and detailed breakdowns be it for academic submissions, portfolio sharing or offline classroom reviews.

Proposed Enhancement:
Implement a client-side document generation feature that allows users to export a comprehensive PDF report summarising their simulation performance.

- **Visual snapshot:** Captures and embeds high resolution image exports directly from the cytoscape.js canvas, illustrating the user's final network spread.
- **Performance and learning analytics:** Format the final score, assigned badge tier (e.g., 🔍 Rookie Detective), parameter settings, and post-by-post breakdown including truth labels and learn-more explainers.

17. Appendix

17.1 References

Independent Cascade Model: https://arxiv.org/html/2405.10322v1

Multi-Criteria Decision Analysis (MCDA) Framework– Weighted Sum Model: https://www.1000minds.com/decision-making/what-is-mcdm-mcda#weighted-sum-models

Google Fact Check API: https://developers.google.com/fact-check/tools/api

17.2 Project Timeline

Milestone 1: Setup & Initialization

- 18 May 2026: Initialize the GitHub repository and finalize the system design, including the end user flow and tech stack
- 20 May 2026: Design Supabase PostgreSQL database schemas and implement Row Level Security (RLS)
- 20 May 2026: Build the foundation of the React.js frontend and connect it to the Supabase backend
- 20 May 2026: Implement user authentication
- 25 May 2026: Display simple graphs for misinformation spread using Cytoscape.js

Milestone 2: Core Logic Implementation

- 6 June 2026: Deployment
- 6 June 2026: Implement the core simulated social media feed populated with curated posts (via Google Fact Check Tools API), enabling interactive actions (like, share, comment, ignore)
- 13 June 2026: Build the core algorithmic spread simulation logic and core scoring formula
- 18 June 2026: Implement playback feature
- 20 June 2026: Integrate the spread simulation logic with real-time dynamic graph representation
- 20 June 2026: Add "learn more" explainers to each post, linking to fact-checks, bias indicators and media literacy tips
- 22 June 2026: Implement onboarding tutorial for first time users
- 22 June 2026: Perform necessary unit testing and integration testing

Milestone 3: Extended Features & Splashdown

- 4 July 2026: Add Contextual Help Icons
- 11 July 2026: Image Classifier Service Expansion
- 11 July 2026: Add Forgot Password flow
- 18 July 2026: Improved Onboarding Tutorial
- 18 July 2026: Polish the UI
- 18 July 2026: Fix outstanding bugs
- 20 July 2026: Post Import Diversity and Feed Variation
- 25 July 2026: Finalize all system documentation

17.3 Contribution Log

Selene

Task	**When**	**Number of hours**
Team meeting	13 May 2026	1
Poster + Figma prototype	14 May 2026	1
Figma prototype + Video	15 May 2026	1
Mission Control #1 React	16 May 2026	2
Mission Control #1 Relational Database	16 May 2026	2
Self learning of Node.js	17 May 2026	2
Self learning of PostgreSQL	17 May 2026	2
Self learning of JWT	17 May 2026	1
Self learning of Git & GitHub	18 May 2026	1
Design database schemas	18 May 2026	2
Backend: Supabase client installation + configuration	18 May 2026	3
Backend: user sign up	19 May 2026	2
Backend: user login	20 May 2026	2
Mission Control #1 Next.js	23 May 2026	2
README update: documentation & development plan	31 May 2026	3
Documentation	4 June 2026	3
Peer review	5 June 2026	3
Google Fact Check API + extracting image from url	5 June 2026	4
Importing posts from scripts to Supabase	6 June 2026	4
Frontend: creating post.ts	6 June 2026	2
Research on graphs visualisations	7 June 2026	2
Research on score formula	7 June 2026	2
Planning and feature refinement	8 June 2026	3
Team meeting: score formula setting + graphs	8 June 2026	3

Vercel deployment trial	9 June 2026	3
Set up dotenv vault to sync .env file	9 June 2026	1
Create attempts table on Supabase + refine the schemas of users and posts	9 June 2026	2
Implementing ICM backend in services/spreadICM.ts	9 June 2026	2
Data cleaning: categorizing truth labels and calculating virality for table posts	9 June 2026	2
Documentation: database validation SQL scripts	9 June 2026	1
Backend: Fetch posts from Supabase	12 June 2026	2
Frontend: learn more explainers	12 June 2026	2
Frontend: sign in button when user is registered	12 June 2026	2
Unit test: fetch post service	12 June 2026	1
Refine MCDA scoring formula	13 June 2026	3
Backend: implement action tracking of users actions	13 June 2026	1
Backend: implement action log and attempt log	13 June 2026	2
Backend: insert attempt log to Supabase attempts	13 June 2026	1
Backend: implement mcda scoring	14 June 2026	3
Backend: implement attemptService to save attempts onto Supabase	14 June 2026	3
Backend: fetch attempts records from Supabase	14 June 2026	2
Backend & Frontend: implement leaderboard sql query and leaderboard page / UI	15 June 2026	2
Unit test: save-attempt route for fetching current logged-in user's id	15 June 2026	2
Unit test: attemptService for saving attempts	15 June 2026	1
Backend: Combine ICM, MCDA and simulation page together	16 June 2026	3
Bug fix: protect URLs and game flow protection	21 June 2026	2
Unit testing: route protection and game flow protection	21 June 2026	2
Team meeting	22 June 2026	2
Feature: onboarding tutorial	22 June 2026	2
Integration test for user flow	23 June 2026	2

Documentation part 14 future enhancements	23 June 2026	2
Video for milestone 2	24 June 2026	2
Documentation part 10 tests	24 June 2026	2
Documentation part 11 - 13 + poster	25 June 2026	2
Bug fix: fetch post unit test	26 June 2026	3
Documentation API	27 June 2026	2
Team meeting	6 Jul 2026	1
Bug fix: duplicate username error during sign in	8 Jul 2026	2
Enhancement: password verification and validation	8 Jul 2026	2
Add stop simulation button	9 Jul 2026	2
Action user state and post state updated	10 Jul 2026	2
Bug: prevent duplicate post actions	10 Jul 2026	2
More sign out buttons	11 Jul 2026	2
Simulation feed variation + image matching	11 Jul 2026	3
Implement score labels	16 Jul 2026	2
Implement loading spinner for buttons	16 Jul 2026	2
Implement logo redirect feature	16 Jul 2026	2
Implement session storage keys for parameters and simulation	16 Jul 2026	3
Added a back to home button in summary	16 Jul 2026	1
Perform walkthrough gate testing	17 Jul 2026	2
Testing: different desktop browsers (firefox, safari, edge, chrome) using playwright	19 Jul 2026	3
User testing form	19 Jul 2026	2
Found and solved a typo in actionLog	22 Jul 2026	1
Milestone 3 documentation: SWE practices + design patterns - Layer diagram for SoC - App Folder Structure	23 Jul 2026	3
Milestone 3 video: script	23 Jul 2026	2
Milestone 3 poster: modify the system design diagram	24 Jul 2026	2

Fix: Reset password confirmation page	24 Jul 2026	3
Milestone 3 documentation: confirm reset password + internal API flow diagrams	24 Jul 2026	2
Milestone 3 video: screen record the app + editing	25 Jul 2026	3

Total number of hours: 166 hours

Valerie

Task	When	Number of hours
Self learning of React + Vite	11 May 2026	3
Team meeting	13 May 2026	1
Self learning of Cytoscape.js	13 May 2026	2
Self learning of Git & GitHub	13 May 2026	1
Poster + Video recording	15 May 2026	1
Mission Control #1 React	16 May 2026	2
Mission Control #1 Git and GitHub	16 May 2026	2
Self learning of Node.js	17 May 2026	1
Self learning of PostgreSQL	17 May 2026	2
Frontend: Build login, signup interface	19 May 2026	1
Frontend: Build simulation interface with posts and user can scroll and view posts	19 May 2026	3
Frontend: Build result and summary interface. User can view how many misinformation they spread and total score they attained.	19 May 2026	2
Frontend: Update simulation interface, fixed repeated interactions problem such that user can only like each post once.	20 May 2026	1
Frontend: Build cytoscape graph. User can view their spread network.	20 May 2026	3
Mission Control #2 Next.js	23 May 2026	2
Frontend: Add live graph visualisation within simulation page such that as user interact with each post, they can see how their spread network changes in real time.	25 May 2026	3
Frontend: Fixed overlapping nodes in graph visualisation such that graph adjusts itself everytime user is added.	25 May 2026	2
Figma prototype + Video	30 May 2026	3
Design diagram. Main user flow that user will go through when the user visits our website.	30 May 2026	2
Update details in README: Add in screenshots of product, furnish details on system design and features.	30 May 2026	1
Poster: Add in system design diagram	30 May 2026	1

Peer evaluation on Skylab	4 June 2026	1
Collate curated post content with corresponding “Learn More” explainers in an excel sheet	4 June 2026	1
Furnish details in documentation: Project Overview, Features & application logic, Design & architecture, UML component diagram & sequence diagram	5 June 2026	3
Research on score formula	6 June 2026	1
Research on spread algorithm	6 June 2026	2
Planning and feature refinement: Researched network propagation models and scoring frameworks, prepared discussion points for team meeting	7 June 2026	1
Team meeting: Settle on score formula, spread logic, task allocation	8 June 2026	2
Backend: Spread logic implemented	9 June 2026	5
Did git branch `deployVercel` for deployment preview on Vercel and the deployment itself. Fix configuration of app for deployment.	9 June 2026	3
Merge deployVercel into main and resolve merge conflicts. Actual deployment of product. Review teammate’s code via pull request.	10 June 2026	1
Signup and login button when clicked does not change to next page. Fix Signup and Login button bug.	10 June 2026	1
Frontend: Redesign simulation page interface – Side by side post and graph	10 June 2026	1
Merge feed/feature into main and resolve merge conflicts. Review teammate’s code via pull request.	12 June 2026	1
Backend: Update ICM Spread logic with updates truth label and activation probability	12 June 2026	1
Unit testing for the ICM spread logic	12 June 2026	3
Frontend: Parametrised scenarios user interface. User can choose follower count (slider), following count (slider), network density (slider), topics (checkbox). Can track what user select as user move on to the simulation page.	12 June 2026	2
Frontend: Redesign user actions – Remove ignore, add dislike.	12 June 2026	1
Frontend: Redesign comment, to include dropbox with comment options for users to choose from. Each comment have different effect, depending on the type of post.	12 June 2026	1
Refine MCDA scoring formula	13 June 2026	1

Merge feed/feed and feature/parameterised-scenario into main and resolve merge conflicts. Review teammate's code via pull request.	13 June 2026	1
Review teammate's code via pull requests. Organise Git Issues and Pull Requests.	14 June 2026	1
Frontend: Implement comment modal pop up, where users can choose which comment and click post to lock in their choice. Enhancement from dropdown box.	14 June 2026	1
Frontend: Redesign network density option to include only 2 choices for users.	14 June 2026	1
Implement learn more panel after summary page, where users learn more about each post (include fact-check link, explanation)	14 June 2026	1
Created new branch test/filter-posts Testing for feature/filter-posts: - Vitest: tested parameter parsing function - Manual testing: Verified single topic filter, multi-topic filter, "No Preference" fallback	14 June 2026	1
Merge feature/feed, feature/learn-more, feature/parameterised-scenario and test/filter-posts into main and resolve merge conflicts.	16 June 2026	1
Integrate cytoscape graph visualisation with ICM spread logic. Connect the backend logic with frontend graph visualisation.	16 June 2026	4
Frontend: (Leaderboard feature) Design leaderboard UI, display rankings, show user position	16 June 2026	1
Created new branch test/scoring Testing for mcda scoring formula: - Vitest: tested calculateA, calculateB, calculateC, calculateFinalScore	16 June 2026	1
Fix: graph layout and leaderboard UI not showing on summary page Add: total reach metrics on summary page	17 June 2026	1
Frontend: Add loading and error states	17 June 2026	1
Mission Control #5	17 June 2026	2
Replay simulation feature	18 June 2026	4
Standardise colour scheme. - Added utils/styles.ts containing the colour tokens for player, follower, and infected states. - Isolated colours from graph-specific logic to allow global reuse across headers, leaderboards etc.	18 June 2026	1
Merge feature/leaderboard, feature/replay and loading-error-state into main and resolve merge conflicts.	20 June 2026	1

Frontend: updated logo and favicon. Update node relationship tags to differentiate follower and following nodes. Update graph visuals to match colour scheme.	20 June 2026	1
Frontend: Polish signup.tsx, login,tsx and home.tsx user interface styling.	20 June 2026	1
Frontend: Polish start.tsx, parameters.tsx and simulation.tsx user interface styling.	21 June 2026	2
Frontend: Redesign score summary page and learn-more page	21 June 2026	1
Frontend: Polish summary.tsx, learn-more.tsx and leaderboard.tsx user interface styling.	21 June 2026	2
Merge ui/style-cleanup and fix deployment error	23 June 2026	1
Team meeting: Discussion on content and workload for documentation, poster, video	23 June 2026	1
Implementing integration testing suite for simulation + ICM + MCDA. Found bugs in simulation.tsx	23 June 2026	4
Fixing bugs in simulation.tsx	24 June 2026	1
Integration testing for simulation + ICM + MCDA	24 June 2026	1
Milestone 2 Poster template	24 June 2026	1
Milestone 2 Video recording, voiceover, combining my part and Selene's part	25 June 2026	1
Add in details for Milestone 2 Poster	25 June 2026	1
README documentation: Part 2. Features and Application Logic Part 3. Design and Architecture Part 9. Deployment	25 June 2026	2
README documentation: Part 4. Simulation Logic Part 10. Testing Strategy Part 13. Maintenance & Troubleshooting Part 15. Appendix	26 June 2026	3
README documentation: Part 6. SWE Practices	27 June 2026	2
GitHub README documentation	27 June 2026	1
Milestone 2 Peer evaluation	2 July 2026	1
Review evaluation from other team, plan tasks for Milestone 3 and create relevant GitHub Issues	6 July 2026	1

Team meeting: Discussion on upcoming plans for Milestone 3 and distribution of workload	7 July 2026	1
Setup continuous integration pipeline. 2 separate jobs for vitest and playwright tests. - automated test run when new PR created and when merge to main	9 July 2026	2
Clearer graph visualisation - smaller nodes - graph legend - different colour nodes for positive and negative activation	9 July 2026	2
Improve graph legend. Initial graph legend covers some part of graph visualisation. Changed it to a modal pop up where user can open and close it as required.	10 July 2026	1
fix(replay feat): - after improving graph visualisation, replay feature only capture final screenshot of the graph - fix: reset nodes back to not yet activated and re-apply activation tick by tick	10 July 2026	1
Add 1 liner for each tick in the replay feature which describes to user what they did and what impact it made	10 July 2026	1
Redesign layout for onboarding tutorial and added clearer UI to improve UI/UX	10 July 2026	2
Implement interactive walkthrough on parameters, simulation and summary page - updated user flow: - user auth → has_seen_onboarding = false? → see onboarding tut → set has_seen_onboarding to true → has_seen_walkthrough = false? → see walkthru on parameters, simulation and summary page → set has_seen_walkthrough to true - add new column has_seen_walkthough to supabase.users table - utilised react joyride to implement walkthrough	13 July 2026	3
Updated simulation integration test, user auth unit test and user-flow e2e test files	13 July 2026	1
Added contextual help icons on parameter page. when the user hovers over the ? icon, explanation of that parameter will appear.	13 July 2026	1
Replay feature: changed play and pause button + slider to next and back button such that user can click next to navigate to next tick and back to navigate to previous tick	13 July 2026	1
Interface Polish (Back Button Improvements)	14 July 2026	1

- Added a reusable Back button that navigates to the previous page using history.state.idx. Will fall back to a configurable route when there's no in-app history to go back to.		
Design and implement navbar with profile - user can easily access to saved simulation (latest 5 attempts), leaderboard, start page and tutorial (if they wish to do tutorial again) - Did a shared tutorial component, which is shared by tutorial.tsx and tutorial-replay.tsx - Did shared navBar component, shared across respective pages	16 July 2026	3
Improved interactive walkthrough UI based on user feedback - timestamp for each step - last button label as “End”, instead of “Last”	16 July 2026	1
Review teammate’s code via pull requests. Resolve merge conflicts and merge feature/navbar and feature/interact-walkthru to main.	17 July 2026	1
Unit testing for saved simulations - tested trend calculations (positive trend, negative trend, zero trend and each attempt only compare immediate neighbour) - tested saved simulation loader	18 July 2026	2
Simulation: allow user to be able to change size of feed and graph canvas depending on own preference	18 July 2026	1
Update UML diagrams to include new details such as has_seen_walkthrough gate etc.	20 July 2026	1
Milestone 3 Documentation Updated: - Features - System architecture	20 July 2026	1
Milestone 3 Poster - Included CI/CD pipeline, updated features segment to include updated screenshots of product, updated future plans	20 July 2026	1
Improve saved simulations UI - added line graph to show trend across attempts - change layout of saved simulations and respective metrics to improve UX - added “View Action Log” such that user can review their actions and impact for each attempt they did – retrieve saved attempts from supabase	21 July 2026	3
Implement dark and light mode - split styles.ts into LIGHT_MODE and DARK_MODE to support theme switching across pages - create app/utils/themeMode.tsx with ThemeProvider and useTheme	21 July 2026	3

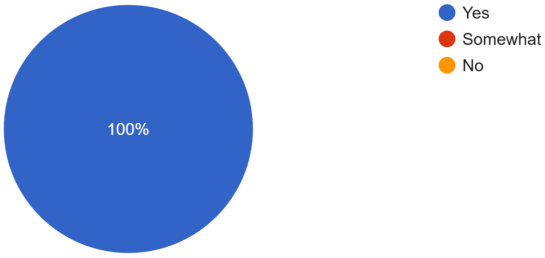
- saved user theme preference in localStorage to persist choices across sessions - Added theme toggle controls to the navigation bar and wrapped the root app in <ThemeProvider>. - Fix tests: wrapped integration tests with <ThemeProvider>, mocked matchMedia and localStorage in Vitest		
Merge and resolve merge conflicts from ui/mode and feature/navbar to main. Update saved-simulation.tsx for dark mode and fix: graph disappears after user zoom out	22 July 2026	2
Milestone 3 Documentation Added: - CI/CD Pipeline	22 July 2026	1
Milestone 3 Documentation + Poster Added: - Future enhancements	24 July 2026	2
Milestone 3 Video - script - voiceover + recording of product	25 July 2026	3
update GitHub README documentation	25 July 2026	1
Milestone 3 Peer evaluation	29 July 2026	2
fix: interaction icons not visible in dark mode	30 July 2026	1
update README documentation to include user testing findings and responses	4 Aug 2026	1

Total number of hours: 169 hours

17.4 User Testing Response

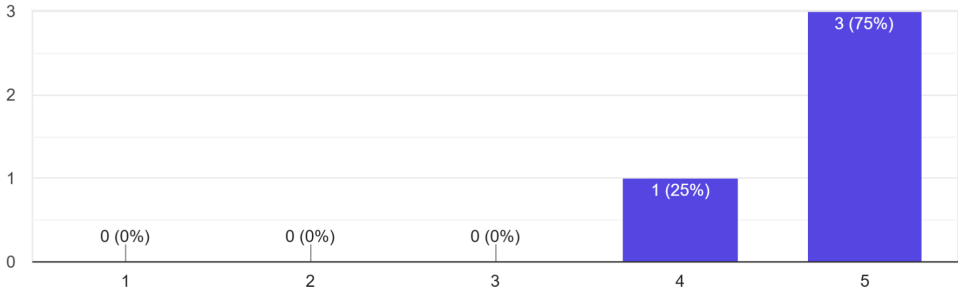
Did you understand the purpose of EchoEcho?

4 responses



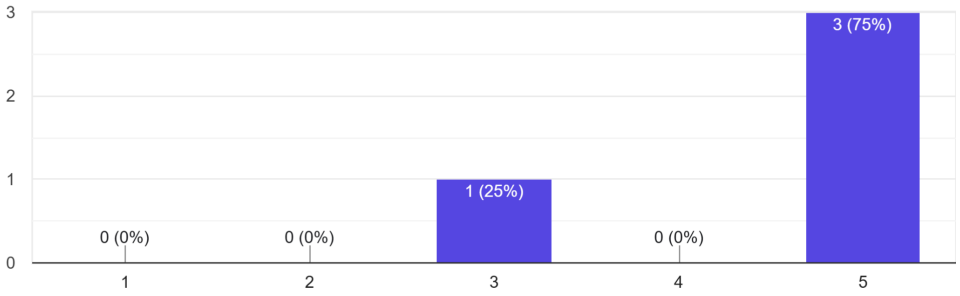
How clear was the app purpose when you first opened it?

4 responses



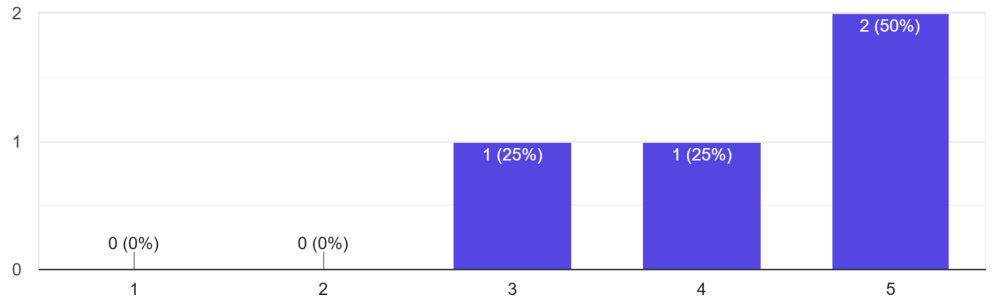
Was the app easy to use overall?

4 responses



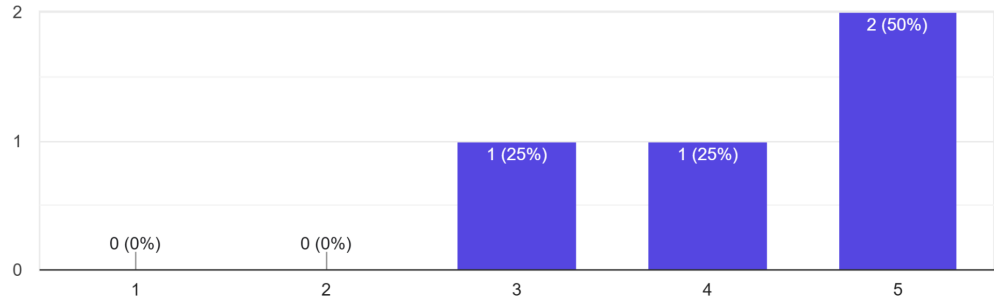
Was the user interface visually clear and easy to understand?

4 responses



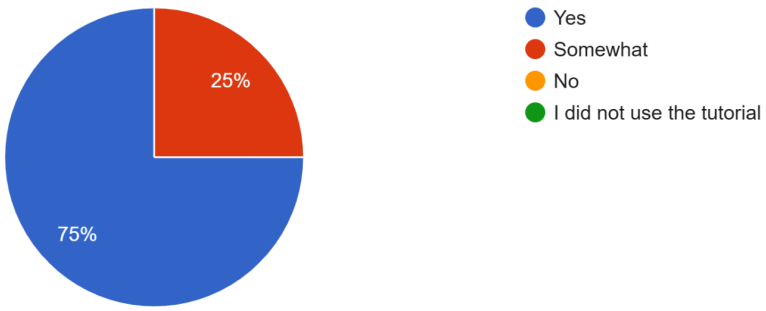
Was the app smooth to use (e.g., page loading, button clicks, simulation flow)?

4 responses



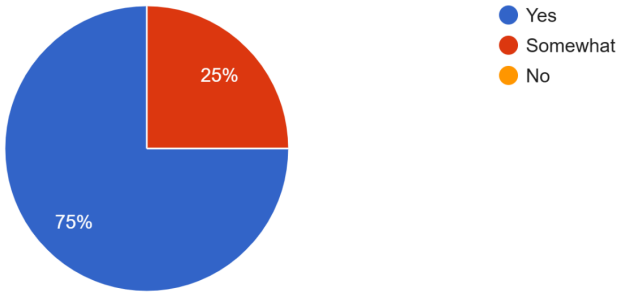
Was the onboarding tutorial helpful?

4 responses



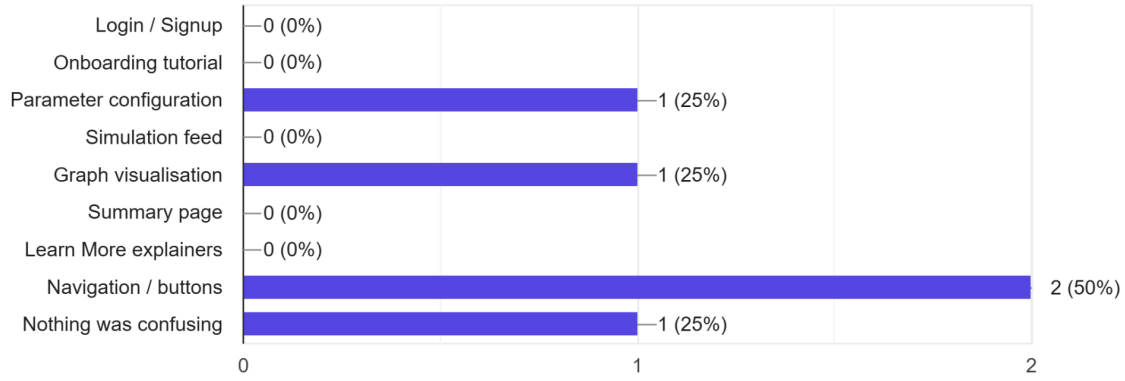
Did you understand how your actions affected the simulation or graph?

4 responses



Which part of the app was most confusing?

4 responses



What did you like most about EchoEcho?

1. The onboarding was really clear. Dividing it into different steps and pointing at the related areas helped me understand what was going on quickly.
2. It's a good reflection on what we usually spread on social media
3. Cool concept and interactivity with the graphs, clean and satisfying UI overall, has potential
4. Clear UI showing clearly how each action will change the spread and the navigation between dark and light mode.

What should we improve?

1. In login/signup, the text is not contrasting with the textbox color well in dark mode, making it a bit hard to see the text.
2. make button clicks smoother
3. Suggestions ranked by priority:
    a. Graph visual clarity:
        i. Problem - Graphs gets cluttered very quickly when increasing the number of nodes, there is also lesser visual clarity between followers and following nodes. Not too sure what the tiny red hexagon next to the node meant either
        ii. Suggestion:
            1. Separate the following and followers nodes to an upper and bottom section, the you node can stay in the middle
            2. Give more distinct and stronger colors to the following and follower nodes, so it can be easily differentiated when the number of nodes increase. Add a legend at the sidebar to indicate which color represents which node to reduce visual clarity
            3. Right now when you click an action button and successfully spread (mis)information, both follower and following nodes get replaced with a single color which becomes difficult to identify which node is which. One option could be to color the arrows and put a colored border around the affected node instead of recoloring the node
            4. Another suggestion is to differentiate correct information and spread of misinformation by marking different colors. So for example by using the colored arrows and borders, if the user spread correct information, the arrows and borders gets marked green, misinformation marks red. This also feeds your users with dopamine which lets them stay longer on your website
            5. If you want to keep that "unknown" factor of not knowing the consequences of your actions, you can first introduce them to the practice colored versions, then suggest them to the testing uncolored version once they are ready. Here the consequences will only show at the end and the colored version can be seen during the propagation playback.
            6. One source of inspiration you can maybe yoink from is Nicky Case: https://ncase.me/projects/ Some of his projects that I find interesting are: The wisdom and/or madness of crowds, We become what we behold (probably the most famous one), The evolution of trust
    b. Activity feed button interaction clarity

i. Problem - There is no clear explanation what each button(like, share, dislike, comment) does
ii. Suggestion:
    1. This is partly solved by having graph visual clarity but it can help more with user interactivity for them to see the immediate positive/negative impacts of each action, rather than a black box where they input a bunch of buttons and somehow output some values
    2. It would also probably help to limit the number of actions a user can do for each piece activity feed, and maybe grey out the feed box once you hit the limit to indicate completion.
    3. Also I dont know if this is an oversight or intended but I can spam share on a single activity feed and keep corrupting further nodes

c. Echo chamber explanation
    i. Problem - Since your graph is initialized at 30 + 15 + 1 nodes, it is difficult to visually spot the difference of how much each % will affect the graph.
    ii. Suggestion - A brief description on the graph creation menu could help

Sorry for the long essay, reminder I'm just a random tester and not everything I say is definitive and correct, so don't take it to heart. A reason why I might have so many ideas could also be because I see the potential in it type shi, so take it as you will. Anyways lmk if anything needs more explaining and all the best!!

4. graph gets somewhat cluttered especially when network becomes dense